



CompTIA

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.

Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

<https://www.CertyIQ.com>

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://www.certyiq.com>

Mail us on - certyiqofficial@gmail.com



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John

October 19, 2022



Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022



Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago



Customer support is really fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022



Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study. Thank You certyiq team!

Henry Rome

2 months ago



These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago



Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's. Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today -3 September with 942 score.

Amazon

(AWS Certified CloudOps Engineer - Associate SOA-C03)

AWS Certified CloudOps Engineer - Associate SOA-C03

Total: **195 Questions**

Link: <https://certyiq.com/papers/amazon/aws-certified-cloudops-engineer-associate-soa-c03>

Question: 1

A CloudOps engineer is examining the following AWS CloudFormation template:

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Creates an EC2 Instance'
Resources:
  EC2Instance:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-79fd7eee
      InstanceType: m5n.large
      SubnetId: subnet-labc3d3fg
      PrivateDnsName: ip-10-24-34-0.ec2.internal
      Tags:
        - Key: Name
          Value: !Sub "${AWS::StackName} Instance"
```

Why will the stack creation fail?

- A. The Outputs section of the CloudFormation template was omitted.
- B. The Parameters section of the CloudFormation template was omitted.
- C. The PrivateDnsName cannot be set from a CloudFormation template.
- D. The VPC was not specified in the CloudFormation template.

Answer: C

Explanation:

C. The PrivateDnsName cannot be set from a CloudFormation template.

This question refers to creating an EC2 instance (or an EC2-based resource) in AWS CloudFormation and attempting to explicitly set a property called:

PrivateDnsName

The key point is:

CloudFormation does NOT allow you to manually set the PrivateDnsName property of an EC2 instance.

Question: 2

A company applies user-defined tags to resources that are associated with the company's AWS workloads. Twenty days after applying the tags, the company notices that it cannot use the tags to filter views in the AWS Cost Explorer console.

What is the reason for this issue?

- A. It takes at least 30 days to be able to use tags to filter views in Cost Explorer.
- B. The company has not activated the user-defined tags for cost allocation.
- C. The company has not created an AWS Cost and Usage Report.
- D. The company has not created a usage budget in AWS Budgets.

Answer: B

Explanation:

The reason the company cannot use tags to filter views in the AWS Cost Explorer console is that user-defined tags must be explicitly activated for cost allocation purposes. AWS differentiates between AWS-generated tags, which are automatically included in billing reports, and user-defined tags. For user-defined tags to be recognized and utilized for cost analysis and filtering within services like Cost Explorer, they require manual activation in the AWS Billing console under the "Cost allocation tags" section.

Without this crucial activation step, AWS Cost Explorer simply does not process these custom tags as dimensions for filtering or grouping costs. Even though the tags were applied to resources for twenty days, their absence from Cost Explorer filters indicates they were not enabled for cost allocation. Once activated, it typically takes 24 to 48 hours for the tag data to propagate and become available for filtering in Cost Explorer. The 20-day period mentioned in the question confirms that the delay is not due to propagation time after activation, but rather the lack of activation itself.

Option A is incorrect because the typical propagation time for tags after activation is much shorter than 30 days. Option C is incorrect as creating an AWS Cost and Usage Report (CUR) provides very granular cost data, but it is not a prerequisite for using activated tags to filter views directly within the Cost Explorer interface. Cost Explorer itself relies on aggregated billing data, which activated tags enrich. Option D, creating a usage budget, is a function of AWS Budgets for cost control and alerts, and has no bearing on enabling tag-based filtering within Cost Explorer. Therefore, activating user-defined tags for cost allocation is the essential missing step.

Authoritative Links:

Using Cost Allocation Tags: <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/cost-alloc-tags.html>

Activating Cost Allocation Tags in the Billing Console:

<https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/activating-tags.html>

Cost Explorer Overview: <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/ce-what-is.html>

Question: 3

CertyIQ

An environment consists of 100 Amazon EC2 Windows instances. The Amazon CloudWatch agent is deployed and running on all EC2 instances with a baseline configuration file to capture log files. There is a new requirement to capture the DHCP log files that exist on 50 of the instances.

What is the MOST operationally efficient way to meet this new requirement?

- A. Create an additional CloudWatch agent configuration file to capture the DHCP logs. Use the AWS Systems Manager Run Command to restart the CloudWatch agent on each EC2 instance with the append-config option to apply the additional configuration file.
- B. Log in to each EC2 instance with administrator rights. Create a PowerShell script to push the needed baseline log files and DHCP log files to CloudWatch.
- C. Run the CloudWatch agent configuration file wizard on each EC2 instance. Verify that the baseline log files are included and add the DHCP log files during the wizard creation process.
- D. Run the CloudWatch agent configuration file wizard on each EC2 instance and select the advanced detail level. This will capture the operating system log files.

Answer: A

Explanation:

Option A is the MOST operationally efficient way to meet this new requirement because it leverages AWS native services for automation and centralized configuration management.

Firstly, creating an additional CloudWatch agent configuration file specifically for DHCP logs promotes modularity and maintainability. This isolates the new requirement without altering the existing baseline configuration, making it easier to manage future changes or revert if necessary.

Secondly, using AWS Systems Manager Run Command is a highly efficient method for executing commands and managing configurations across a fleet of EC2 instances at scale. Instead of manually logging into each of the 50 instances, Run Command allows for a single action to be applied to all targeted instances simultaneously and programmatically. This drastically reduces manual effort, potential for human error, and operational overhead.

The append-config option for the CloudWatch agent is crucial here. It allows the agent to merge the new DHCP log configuration with the existing baseline configuration already running on the instances. This means the existing log collection continues uninterrupted while the new DHCP log collection is seamlessly added, ensuring no downtime or loss of existing data streams. This approach avoids the need to consolidate all log configurations into a single, potentially complex, file or to restart collection of baseline logs unnecessarily.

In contrast, options B, C, and D involve manual intervention on each EC2 instance (logging in or running a wizard), which is incredibly inefficient and error-prone for 50 instances, let alone a larger fleet. Option B also suggests abandoning the robust CloudWatch agent for custom PowerShell scripts, which would increase maintenance burden and re-implement functionality already provided by the agent. Options C and D involve interacting with a wizard, which is not designed for programmatic, large-scale deployment or updates in an automated environment. Moreover, option D's "advanced detail level" might collect more data than necessary and doesn't guarantee specific DHCP log capture without explicit configuration, nor does it address the deployment efficiency.

Therefore, Option A aligns with best practices for infrastructure automation and centralized management using AWS services, ensuring minimal operational effort and maximum efficiency for updating log collection requirements.

Authoritative Links for further research:

Deploy the CloudWatch agent using Systems Manager:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/create-cloudwatch-agent-configuration-file-sagemaker.html>

Run Commands using AWS Systems Manager: <https://docs.aws.amazon.com/systems-manager/latest/userguide/execute-remote-commands.html>

CloudWatch Agent Configuration File Structure:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Agent-Configuration-File-Details.html> (Look for append_dimensions and related options in the context of configuration management, although append-config is a command-line option for deployment, it signifies the merging capability.)

Question: 4

CertyIQ

A company is storing backups in an Amazon S3 bucket. The backups must not be deleted for at least 3 months after the backups are created.

What should a CloudOps engineer do to meet this requirement?

- A. Configure an IAM policy that denies the s3:DeleteObject action for all users. Three months after an object is written, remove the policy.
- B. Enable S3 Object Lock on a new S3 bucket in compliance mode. Place all backups in the new S3 bucket with a retention period of 3 months.

- C. Enable S3 Versioning on the existing S3 bucket. Configure S3 Lifecycle rules to protect the backups.
- D. Enable S3 Object Lock on a new S3 bucket in governance mode. Place all backups in the new S3 bucket with a retention period of 3 months.

Answer: B

Explanation:

The correct solution is **B. Enable S3 Object Lock on a new S3 bucket in compliance mode. Place all backups in the new S3 bucket with a retention period of 3 months.**

Here's a detailed justification:

The core requirement is to ensure backups "must not be deleted for at least 3 months" after creation, implying an immutable, write-once-read-many (WORM) storage mechanism for a specified duration. S3 Object Lock is specifically designed for this purpose, providing a robust layer of data protection against accidental or malicious deletion and modification.

Option B is correct because:

1. **S3 Object Lock:** This feature provides WORM capability for S3 objects, preventing them from being deleted or overwritten for a fixed amount of time or indefinitely.
2. **Compliance Mode:** In compliance mode, an object version cannot be overwritten or deleted by any user, including the root account, until the retention period expires. This offers the highest level of protection and is ideal for scenarios where data immutability is paramount, such as regulatory compliance or critical backup retention.
3. **Retention Period:** Setting a retention period of 3 months directly satisfies the specified duration for non-deletion. Once the period is set in compliance mode, it cannot be shortened or disabled.
4. **New S3 Bucket:** S3 Object Lock must be enabled at the bucket level when the bucket is created. If the existing bucket doesn't have it, a new bucket is necessary to implement this solution.

Why other options are incorrect:

A. Configure an IAM policy that denies the s3:DeleteObject action for all users. Three months after an object is written, remove the policy.

IAM policies control user permissions, not the intrinsic immutability of the object itself. While an IAM policy can prevent most users from deleting objects, a malicious actor with sufficient permissions (e.g., the ability to modify IAM policies or the root user) could bypass it.

The manual process of removing the policy for each object or batch after 3 months is operationally complex, prone to errors, and not scalable for a backup system.

C. Enable S3 Versioning on the existing S3 bucket. Configure S3 Lifecycle rules to protect the backups.

S3 Versioning helps protect against accidental overwrites by keeping multiple versions of an object.

However, it does not prevent the deletion of specific object versions or the entire object if a user has the necessary permissions. A user could still explicitly delete all versions.

S3 Lifecycle rules are primarily used for managing object storage classes and expiring (deleting) objects after a specified period. They facilitate data deletion or transition, rather than preventing deletion for a minimum duration. While you could set a rule to expire objects after 3 months, this is the opposite of preventing deletion and does not provide an immutable lock. There's no lifecycle rule to enforce a "cannot be deleted" state.

D. Enable S3 Object Lock on a new S3 bucket in governance mode. Place all backups in the new S3 bucket with a retention period of 3 months.

Governance Mode of S3 Object Lock prevents most users from deleting or overwriting an object version for the retention period. However, users with specific permissions (s3:BypassGovernanceRetention) or the root user can still modify or delete objects even when a retention period is active.

The requirement states "must not be deleted," implying a strong, unbypassable lock. Governance mode, while

strong, is not as absolute as compliance mode and thus does not fully meet the stringent "must not be deleted" requirement if a powerful user intends to bypass it.

Therefore, enabling S3 Object Lock in compliance mode is the most robust and appropriate solution to meet the requirement of ensuring backups are not deleted for at least 3 months.

Authoritative Links for further research:

S3 Object Lock Overview: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>

Object Lock Modes: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock-overview.html#object-lock-modes>

Managing S3 Versioning: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html>

Managing your storage lifecycle: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-lifecycle-mgt.html>

Question: 5

CertyIQ

A company's CloudOps engineer is troubleshooting communication between the components of an application. The company configured VPC flow logs to be published to Amazon CloudWatch Logs. However, there are no logs in CloudWatch Logs.

What could be blocking the VPC flow logs from being published to CloudWatch Logs?

- A. The IAM policy that is attached to the IAM role for the flow log is missing the logs:CreateLogGroup permission.
- B. The IAM policy that is attached to the IAM role for the flow log is missing the logs:CreateExportTask permission.
- C. The VPC is configured for IPv6 addresses.
- D. The VPC is peered with another VPC in the AWS account

Answer: A

Explanation:

The issue described, where VPC flow logs are configured but no logs appear in CloudWatch Logs, strongly indicates a permissions problem preventing the log group or log streams from being created or updated. When setting up VPC Flow Logs to publish to CloudWatch Logs, AWS requires an IAM role with specific permissions for the flow log service to interact with CloudWatch.

Option A is the correct answer because the IAM role used by VPC Flow Logs to publish to CloudWatch Logs must have logs:CreateLogGroup permission. This permission allows the service to create the designated log group in CloudWatch Logs if it doesn't already exist. If this fundamental permission is missing, the flow log service cannot establish the necessary destination for the logs, resulting in no log data being published whatsoever. Additionally, the role also requires logs:PutLogEvents and logs:CreateLogStream permissions to write events into log streams within that log group and create new streams as needed. However, CreateLogGroup is the critical permission for the initial setup if the log group itself doesn't exist, directly leading to the "no logs" symptom.

Let's evaluate why the other options are incorrect:

B. The IAM policy that is attached to the IAM role for the flow log is missing the logs:CreateExportTask permission. The logs:CreateExportTask permission is used for exporting existing log data from CloudWatch Logs to an S3 bucket. It is not related to the initial process of publishing VPC flow logs to CloudWatch Logs. Therefore, its absence would not prevent logs from appearing in CloudWatch Logs in the first place.

C. The VPC is configured for IPv6 addresses. VPC Flow Logs support capturing traffic for both IPv4 and IPv6 addresses. The presence of IPv6 configuration does not inherently block flow logs from being published; it merely means that flow logs will include IPv6 traffic alongside IPv4 traffic, if applicable.

D. The VPC is peered with another VPC in the AWS account. VPC peering allows communication between two VPCs but does not interfere with the functionality of VPC Flow Logs for either VPC. Flow logs capture traffic flowing within and to/from the configured VPC, regardless of peering connections. The mechanism for publishing these logs to CloudWatch Logs remains unaffected by VPC peering.

In summary, for VPC Flow Logs to successfully publish to CloudWatch Logs, the associated IAM role must possess the necessary permissions to interact with CloudWatch Logs, including `logs:CreateLogGroup` to create the log group if it doesn't exist, and `logs:PutLogEvents` to write the actual log data. The absence of `logs:CreateLogGroup` is a direct blocker for the initial logging setup.

Authoritative Links for further research:

Publishing Flow Logs to CloudWatch Logs:

<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs-cwl.html>

IAM Role for Flow Logs: This document details the specific permissions required for the IAM role.

<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs-cwl.html#flow-logs-cwl-iam>

Troubleshooting VPC Flow Logs:

<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs-troubleshooting.html>

Question: 6

CertyIQ

A company is migrating a legacy application to AWS. The company manually installs and configures the legacy application on Amazon EC2 instances across multiple Availability Zones. The company sets up an Application Load Balancer (ALB) for the application. The company sets the target group routing algorithm to weighted random. The application requires session affinity.

After the company deploys the application, users report random application errors that were not present in the legacy version of the application. The target group health checks do not show any failures. The company must resolve the application errors.

Which solution will meet this requirement?

- A. Set the routing algorithm of the target group to least outstanding requests.
- B. Turn on anomaly mitigation for the target group.
- C. Turn off the cross-zone load balancing attribute of the target group.
- D. Increase the deregistration delay attribute of the target group.

Answer: A

Explanation:

The company's legacy application explicitly requires session affinity, meaning that requests from a single user's session should ideally be routed to the same Amazon EC2 instance to maintain session-specific data. The current configuration uses an Application Load Balancer (ALB) with the target group routing algorithm set to weighted random. This algorithm distributes incoming requests among healthy targets based on their assigned weights, with no consideration for preserving a user's session. Consequently, consecutive requests from the same user are frequently routed to different EC2 instances, causing the application to lose session state and resulting in the reported "random application errors." The fact that health checks are passing indicates the instances themselves are functional, but the application logic breaks due to the routing strategy.

Changing the routing algorithm to least outstanding requests (Option A) is the most effective solution among the provided choices to mitigate these issues. The least outstanding requests algorithm directs new requests to

the target that currently has the fewest requests in progress. While this algorithm does not explicitly provide true session stickiness (which would typically be enabled through the `stickiness.enabled` attribute of the target group), it offers a more stable and less dispersive routing pattern compared to weighted random.

With weighted random, requests are distributed purely probabilistically, maximizing the chance of a user's session being split across different instances. In contrast, least outstanding requests attempts to balance the load evenly by always sending requests to the least busy server. This deterministic approach can inadvertently increase the likelihood that a rapid sequence of requests from the same user arrives at the same instance, especially if that instance remains the least busy for a short duration. By reducing the frequency of server switches for a given user, this strategy can significantly reduce the occurrence of broken sessions and, consequently, the "random application errors" that stem from a lack of session affinity, even if it doesn't guarantee stickiness.

Let's briefly examine why other options are less suitable:

B. Turn on anomaly mitigation for the target group: Anomaly mitigation focuses on proactively identifying and removing misbehaving instances before they fail health checks. This addresses instance health and reliability, not the specific problem of session affinity breaking due to routing.

C. Turn off the cross-zone load balancing attribute of the target group: This attribute determines whether ALB nodes route requests to targets across all Availability Zones or only within their own AZ. Turning it off can lead to uneven load distribution and does not resolve the core issue of session affinity across instances.

D. Increase the deregistration delay attribute of the target group: Deregistration delay (connection draining) allows existing connections to complete gracefully when an instance is deregistering or becoming unhealthy. This is crucial for application availability during scaling or maintenance but has no bearing on maintaining session affinity for active, healthy instances under normal operation.

Therefore, by selecting least outstanding requests, the company can achieve a more stable request distribution that, while not a perfect session stickiness solution, significantly improves session continuity compared to weighted random routing, thereby mitigating the reported application errors.

Authoritative Links for Further Research:

Application Load Balancer Target Groups and Routing Algorithms:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html>

(Refer to `load_balancing.algorithm.type` attribute)

Session Stickiness for Application Load Balancers:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html#sticky-sessions>

Question: 7

CertyIQ

A company is using an Amazon Aurora MySQL DB cluster that has point-in-time recovery, backtracking, and automatic backup enabled. A CloudOps engineer needs to be able to roll back the DB cluster to a specific recovery point within the previous 72 hours. Restores must be completed in the same production DB cluster. Which solution will meet these requirements?

- A. Create an Aurora Replica. Promote the replica to replace the primary DB instance.
- B. Create an AWS Lambda function to restore an automatic backup to the existing DB cluster.
- C. Use backtracking to rewind the existing DB cluster to the desired recovery point.
- D. Use point-in-time recovery to restore the existing DB cluster to the desired recovery point.

Answer: C

Explanation:

The correct solution is **C. Use backtracking to rewind the existing DB cluster to the desired recovery point.**

Here's a detailed justification:

1. **Understanding Aurora Backtracking:** Amazon Aurora Backtracking is a unique feature specifically designed to quickly rewind an Amazon Aurora DB cluster to a prior point in time in place. It effectively allows you to "undo" recent database changes without creating a new DB cluster or losing the current cluster endpoint.
2. **Meeting the "Same Production DB Cluster" Requirement:** This is the most critical constraint in the question. The problem explicitly states, "Restores must be completed in the same production DB cluster." Aurora Backtracking is the only option provided that performs an in-place rewind of the existing cluster. Other recovery methods for Aurora, such as Point-in-Time Recovery or restoring from snapshots, always create a new DB cluster.
3. **Leveraging Enabled Features:** The question states that "backtracking" is enabled, indicating that the environment is already configured to utilize this capability. This makes it a viable and intended solution.
4. **Time Window Compatibility:** The requirement to roll back within the "previous 72 hours" aligns perfectly with Aurora Backtracking's capabilities. Backtracking windows typically support up to 72 hours, providing ample flexibility within the specified timeframe.
5. **Speed and Efficiency:** Backtracking is significantly faster than traditional point-in-time recovery because it doesn't involve moving data. Instead, Aurora changes its logical view of the data to reflect a previous state by discarding changes that occurred after the target rewind point. This makes it ideal for rapid recovery from accidental data modifications or application errors.
6. **Why other options are incorrect:**
 - A. Create an Aurora Replica. Promote the replica to replace the primary DB instance:** While an Aurora Replica could potentially have older data if created before an issue, promoting it creates a new primary cluster from the replica. It does not rewind or modify the original production DB cluster in place. This fails the "same production DB cluster" requirement.
 - B. Create an AWS Lambda function to restore an automatic backup to the existing DB cluster:** Restoring from an automatic backup (or a manual snapshot) in Aurora always creates a new DB cluster. You cannot restore an automatic backup onto an existing cluster to overwrite or rewind it. This violates the "same production DB cluster" constraint.
 - D. Use point-in-time recovery to restore the existing DB cluster to the desired recovery point:** Point-in-Time Recovery (PITR) in Aurora allows you to restore to a specific time, but it, too, creates a new DB cluster with the restored data. It does not perform an in-place rollback of the original cluster. Thus, it fails the critical requirement of completing the restore "in the same production DB cluster."

In summary, Aurora Backtracking is the precise feature designed for efficient, in-place recovery of an Aurora DB cluster to a prior state, directly fulfilling all conditions outlined in the problem statement.

Authoritative Links for Further Research:

Rewinding an Aurora DB cluster with

Backtrack:<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Managing.Backtrack.html>

Backtracking Aurora DB clusters:<https://aws.amazon.com/blogs/aws/new-amazon-aurora-backtrack-rewind-your-db-cluster-to-an-earlier-point-in-time/>

Question: 8

A CloudOps engineer is troubleshooting an AWS CloudFormation stack creation that failed. Before the CloudOps engineer can identify the problem, the stack and its resources are deleted. For future deployments, the CloudOps engineer must preserve any resources that CloudFormation successfully created.

What should the CloudOps engineer do to meet this requirement?

- A. Set the value of the DisableRollback parameter to False during stack creation.
- B. Set the value of the OnFailure parameter to DO_NOTHING during stack creation.
- C. Specify a rollback configuration that has a rollback trigger of DO_NOTHING during stack creation
- D. Set the value of the OnFailure parameter to ROLLBACK during stack creation.

Answer: B

Explanation:

When an AWS CloudFormation stack creation fails, the default behavior is for CloudFormation to initiate a rollback, deleting any resources that were successfully created during the stack creation process. This default action, while ensuring a clean state, makes it challenging to troubleshoot the root cause of the failure, as the resources that might hold clues are removed.

To meet the requirement of preserving successfully created resources for future troubleshooting, the CloudOps engineer should set the value of the OnFailure parameter to DO_NOTHING during stack creation. This parameter dictates CloudFormation's behavior when a stack operation, such as creation, fails.

By setting OnFailure to DO_NOTHING, CloudFormation will halt the creation process at the point of failure and will not attempt to roll back or delete any resources that were successfully provisioned up to that point. Instead, the stack will remain in a CREATE_FAILED state, and all partially created resources will persist. This allows the engineer to inspect the existing resources, logs, and configurations to identify the exact problem without losing valuable diagnostic information.

In contrast, setting OnFailure to ROLLBACK (the default behavior) would result in the deletion of all successfully created resources, which directly contradicts the requirement. Similarly, setting OnFailure to DELETE would also remove the stack and its resources.

Option A, setting DisableRollback to False, is incorrect because False is the default state, meaning rollback is enabled, leading to resource deletion. If DisableRollback were set to True, it would prevent rollback, but OnFailure is the more specific and recommended parameter for controlling failure actions. Option C is incorrect because rollback triggers are for monitoring and initiating rollbacks after a stack is operational, and "DO_NOTHING" is not a valid trigger type. Therefore, OnFailure=DO_NOTHING is the precise solution for preserving resources upon stack creation failure.

For further research:

AWS CloudFormation OnFailure parameter:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/parameters-section-stack-failure.html>

Controlling rollback behavior: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-parameters.html#parameters-features-disable-rollback>

Question: 9

A company plans to run a public web application on Amazon EC2 instances behind an Elastic Load Balancing (ELB) load balancer. The company's security team wants to protect the website by using AWS Certificate Manager

(ACM) certificates. The load balancer must automatically redirect any HTTP requests to HTTPS. Which solution will meet these requirements?

- A. Create an Application Load Balancer that has one HTTPS listener on port 80. Attach an SSL/TLS certificate to listener port 80. Create a rule to redirect requests from HTTP to HTTPS.
- B. Create an Application Load Balancer that has one HTTP listener on port 80 and one HTTPS protocol listener on port 443. Attach an SSL/TLS certificate to listener port 443. Create a rule to redirect requests from port 80 to port 443.
- C. Create an Application Load Balancer that has two TCP listeners on port 80 and port 443. Attach an SSL/TLS certificate to listener port 443. Create a rule to redirect requests from port 80 to port 443.
- D. Create a Network Load Balancer that has two TCP listeners on port 80 and port 443. Attach an SSL/TLS certificate to listener port 443. Create a rule to redirect requests from port 80 to port 443.

Answer: B

Explanation:

Option B presents the correct and most efficient solution for configuring an Application Load Balancer (ALB) to meet the specified requirements. Application Load Balancers operate at Layer 7 of the OSI model, making them ideal for HTTP and HTTPS traffic management, including SSL/TLS termination and content-based routing.

Here's a detailed justification:

1. **Application Load Balancer Choice:** An ALB is the appropriate load balancer type because it supports advanced HTTP/HTTPS features required for this scenario, such as SSL termination and native redirection. Network Load Balancers (NLBs) operate at Layer 4 and lack these application-layer capabilities.
2. **HTTP Listener on Port 80:** The ALB must have an HTTP listener configured on port 80. This listener is responsible for accepting incoming unencrypted HTTP requests from clients attempting to access the website.
3. **HTTPS Listener on Port 443:** Concurrently, an HTTPS listener must be configured on port 443. This listener handles secure, encrypted HTTPS requests. Port 443 is the standard port for HTTPS traffic, ensuring compatibility with web browsers and clients.
4. **SSL/TLS Certificate Attachment:** The AWS Certificate Manager (ACM) certificate, which secures the website, is attached directly to the HTTPS listener on port 443. This action enables SSL/TLS termination at the load balancer, offloading the encryption/decryption process from the backend EC2 instances.
5. **Native HTTP-to-HTTPS Redirection:** A key feature of Application Load Balancers is their ability to define rules for listeners. A rule can be created for the HTTP listener on port 80 to automatically redirect all incoming requests to the HTTPS listener on port 443. This ensures that any user trying to access the website via an unencrypted HTTP connection is seamlessly and securely rerouted to the HTTPS version of the site, fulfilling the security team's requirement.
6. **Why other options are incorrect:**

Option A is incorrect because an HTTPS listener must be on port 443, not 80, for standard web operation. Attaching an SSL certificate to an HTTPS listener on port 80 is a misconfiguration that will prevent proper secure communication.

Option C specifies "TCP listeners" for an ALB. While ALBs support some TCP passthrough, their primary value comes from HTTP/HTTPS listeners that enable Layer 7 features like redirection. Using generic TCP listeners would bypass the ALB's native application-layer intelligence for HTTP/HTTPS handling and redirection.

Option D suggests a Network Load Balancer (NLB). NLBs operate at Layer 4 and do not natively support application-layer HTTP-to-HTTPS redirection or SSL termination at the load balancer itself in the same way an ALB does (though they can support TLS listeners, the redirection logic is absent). Implementing such

redirection with an NLB would require configuration on the backend instances, which contradicts the requirement for the load balancer to handle it automatically.

Therefore, creating both an HTTP listener on port 80 and an HTTPS listener on port 443, attaching the ACM certificate to the HTTPS listener, and setting up a redirection rule from port 80 to port 443 on an Application Load Balancer is the correct, secure, and most efficient solution.

Authoritative Links for Further Research:

Application Load Balancer Listeners:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/listeners.html>

Redirect actions for your Application Load Balancer:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/listener-update-rules.html#redirect-actions>

SSL/TLS certificates for Application Load Balancers:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-https-listener.html>

What is AWS Certificate Manager (ACM)?

<https://aws.amazon.com/certificate-manager/>

Question: 10

CertyIQ

A company uses AWS Organizations to manage a set of AWS accounts. The company has set up organizational units (OUs) in the organization. An application OU supports various applications.

A CloudOps engineer must prevent users from launching Amazon EC2 instances that do not have a CostCenter-Project tag into any account in the application OU. The restriction must apply only to accounts in the application OU.

Which solution will meet these requirements?

- A. Create an IAM group that has a policy that allows the `ec2:RunInstances` action when the `CostCenter-Project` tag is present. Place all IAM users who need access to the application accounts in the IAM group.
- B. Create a service control policy (SCP) that denies the `ec2:RunInstances` action when the `CostCenter-Project` tag is missing. Attach the SCP to the application OU.
- C. Create an IAM role that has a policy that allows the `ec2:RunInstances` action when the `CostCenter-Project` tag is present. Attach the IAM role to the IAM users that are in the application OU accounts.
- D. Create a service control policy (SCP) that denies the `ec2:RunInstances` action when the `CostCenter-Project` tag is missing. Attach the SCP to the root OU.

Answer: B

Explanation:

The correct solution is **B. Create a service control policy (SCP) that denies the `ec2:RunInstances` action when the `CostCenter-Project` tag is missing. Attach the SCP to the application OU.**

Here's a detailed justification:

1. **Service Control Policies (SCPs) for Centralized Governance:** AWS Organizations allows you to centrally manage and govern your environment. SCPs are powerful guardrails that define the maximum available permissions for accounts within an Organizational Unit (OU) or the entire organization. They are ideal for enforcing mandatory controls across multiple accounts.
2. **Explicit Deny for Prevention:** The requirement is to "prevent" users from launching EC2 instances without a specific tag. An SCP with an explicit Deny statement is the most effective way to achieve this. An explicit deny in an SCP overrides any Allow policies that might exist at the IAM user, group, or

role level within the affected accounts, ensuring the restriction is enforced absolutely.

3. **Conditional Denial based on Tags:** SCPs can utilize conditions to evaluate requests. In this scenario, the `ec2:RunInstances` action needs to be denied when the `CostCenter-Project` tag is missing. This can be achieved using a condition key like `aws:RequestTag/CostCenter-Project` combined with the `Null:true` operator in the SCP, which checks if the tag key is not provided in the request.
4. **Targeted Scope to the Application OU:** The problem statement explicitly states, "The restriction must apply only to accounts in the application OU." By attaching the SCP directly to the application OU, the policy's effects are confined to the accounts within that specific OU, precisely meeting this scope requirement without impacting other OUs or accounts in the organization.
5. **Why other options are less suitable:**

A and C (IAM policies/roles): While IAM policies can enforce tag requirements, they are permissions within an account. They rely on meticulous management of every user and role's permissions across potentially many accounts. If a user has another IAM policy or role that grants `ec2:RunInstances` without the tag condition, this solution can be bypassed. It's not a centralized, overriding preventative measure.

D (SCP attached to root OU): Attaching the SCP to the root OU would apply the restriction to all accounts in the organization, including those outside the application OU. This violates the requirement that the restriction apply only to accounts in the application OU.

In summary, an SCP provides the necessary centralized control, explicit denial capability, and precise scoping to meet all the requirements for preventing untagged EC2 instance launches in the specified OU.

Authoritative Links for Further Research:

Service Control Policies (SCPs):

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scp.html

Controlling access to AWS resources using tags:

https://docs.aws.amazon.com/IAM/latest/UserGuide/access_tags.html

AWS Global Condition Context Keys (specifically `aws:RequestTag`):

https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_condition-keys.html#condition-keys-requesttag

Question: 11

CertyIQ

A company runs a business application on more than 300 Linux-based instances. Each instance has the AWS Systems Manager Agent (SSM Agent) installed. The company expects the number of instances to grow in the future. All business application instances have the same user-defined tag.

A CloudOps engineer wants to run a command on all the business application instances to download and install a package from a private repository. To avoid overwhelming the repository, the CloudOps engineer wants to ensure that no more than 30 downloads occur at one time.

Which solution will meet this requirement in the MOST operationally efficient way?

- A. Use a secondary tag to create 10 batches of 30 instances each. Use a Systems Manager Run Command document to download and install the package. Specify the target as part of the Run Command document by using the secondary tag. Run each batch one time.
- B. Use an AWS Lambda function to automatically run a Systems Manager Run Command document that reads a list of instance IDs that have the user-defined tag. Set reserved concurrency for the Lambda function to 30.
- C. Use a Systems Manager Run Command document to download and install the package. Use rate control to set concurrency to 30. Specify the target by using the user-defined tag as part of the Run Command document.
- D. Use a parallel workflow state in AWS Step Functions to automatically run a Systems Manager Run Command document that reads a list of instance IDs that have the user-defined tag. Set the number of parallel states to

30. Run the Step Functions workflow 10 times.

Answer: C

Explanation:

Option C is the most operationally efficient solution because it leverages the native capabilities of AWS Systems Manager Run Command, which is specifically designed for this type of task.

1. **Systems Manager Run Command for Execution:** AWS Systems Manager Run Command is the ideal service for executing commands, scripts, or Systems Manager documents on a fleet of EC2 instances. It ensures the command runs reliably across all targeted instances.
2. **Built-in Rate Control (Concurrency):** Run Command features a "Rate Control" option that allows specifying both a "Concurrency" limit and an "Error Threshold." By setting the concurrency to 30, the CloudOps engineer directly addresses the requirement to ensure no more than 30 downloads occur at one time, preventing the private repository from being overwhelmed. Systems Manager automatically manages the scheduling and execution of the command across the instances, adhering to this limit. This eliminates the need for manual batching or external orchestration to enforce the concurrency.
3. **Tag-Based Targeting:** Run Command supports targeting instances dynamically using user-defined tags. Since all business application instances have the same user-defined tag, the CloudOps engineer can simply specify this tag as the target. This makes the solution highly scalable and future-proof; as new instances are launched with this tag, they are automatically included in subsequent Run Command executions without any modification to the command itself.
4. **Operational Efficiency:** This approach requires only a single Run Command execution. Systems Manager handles all the underlying complexity of instance discovery (via tags), command distribution, concurrency management, and reporting. This minimizes manual intervention, custom code development (like Lambda or Step Functions logic for batching), and the management overhead associated with complex orchestration. It's a "set-and-forget" mechanism for this specific requirement.

Let's briefly look at why other options are less efficient:

A. Secondary Tagging and Multiple Batches: This approach introduces significant manual operational overhead. Creating and managing 10 secondary tags for 300 instances, and then running 10 separate Run Commands, is cumbersome and error-prone. As the instance count grows, this manual tagging and multiple execution strategy becomes unsustainable and highly inefficient.

B. AWS Lambda for Orchestration: While Lambda can invoke Run Command, setting Lambda's reserved concurrency to 30 doesn't directly control the Run Command's operational concurrency for package downloads across 300+ instances without complex custom logic within the Lambda function to invoke Run Command for individual instances or small batches, managing state, and ensuring the actual download limit. Systems Manager's native concurrency control is a much simpler and more direct solution. This adds unnecessary complexity and cost.

D. AWS Step Functions with Parallel States: Step Functions is powerful for complex workflows, but for a task like this with native Run Command support, it's over-engineering. Setting 30 parallel states and then running the workflow 10 times is highly inefficient and cumbersome. A single Run Command execution with built-in rate control achieves the same goal with significantly less setup, management, and execution complexity.

In summary, Option C leverages Systems Manager's purpose-built features for remote execution, dynamic targeting, and most importantly, native rate control, making it the most operationally efficient, scalable, and cost-effective solution for this scenario.

Authoritative Links for further research:

AWS Systems Manager Run Command:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/execute-remote-commands.html>

Working with Concurrency and Error Thresholds (Rate Control) in Run Command:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/send-commands-about-rate-control.html>

Targeting instances in Run Command:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/send-commands-target.html>

Question: 12

CertyIQ

A company uses Amazon Route 53 with latency-based routing across multiple AWS Regions to provide resiliency. The company uses Route 53 with latency-based routing to direct traffic to the nearest Region. Within each Region, weighted A records distribute traffic across multiple Availability Zones. During a recent update, some Availability Zone endpoints became unhealthy. Route 53 continued to route traffic to the unhealthy endpoints. The company must prevent this issue from occurring in the future. Which solution will meet this requirement?

- A. Add a Route 53 health check for each of the weighted records that received traffic during the recent update.
- B. Increase the weight of Route 53 records in the Region where traffic must go during updates.
- C. Reconfigure all records to use latency-based routing across all Regions uniformly.
- D. Reduce the TTL value for latency-based routing to detect changes more quickly.

Answer: A

Explanation:

The core problem described is that Amazon Route 53 continued directing traffic to Availability Zone (AZ) endpoints within a Region even after they became unhealthy during an update. This indicates a lack of automated health detection and failover at the granular level where traffic is distributed within a Region.

Option A: Add a Route 53 health check for each of the weighted records that received traffic during the recent update. This is the correct solution because Route 53 health checks are specifically designed to monitor the health and performance of endpoints that are configured with DNS records. When a health check is associated with a weighted A record, Route 53 continuously monitors the status of the resource (e.g., an EC2 instance, application endpoint, or load balancer) behind that record. If the health check determines that the endpoint is unhealthy, Route 53 automatically stops including that record in its DNS responses. This ensures that user traffic is only routed to healthy endpoints, thereby preventing service disruptions caused by routing to failed resources. In this scenario, adding health checks to the weighted A records (which distribute traffic across multiple Availability Zones) would enable Route 53 to detect when an AZ endpoint becomes unhealthy and automatically remove it from rotation, directing traffic to the remaining healthy AZs. This directly addresses the requirement to prevent routing traffic to unhealthy endpoints in the future, enhancing the company's fault tolerance and resilience.

Why other options are incorrect:

B. Increase the weight of Route 53 records in the Region where traffic must go during updates. Increasing the weight on a record would direct more traffic to it, not less. This would only exacerbate the problem if the endpoint is unhealthy, unless combined with a health check to dynamically adjust routing. It does not provide a mechanism to detect unhealthiness.

C. Reconfigure all records to use latency-based routing across all Regions uniformly. The company already uses latency-based routing across Regions. The issue occurred within a Region, at the AZ level, due to unhealthy endpoints. Reconfiguring routing uniformly would not introduce a mechanism to detect endpoint health or prevent routing to unhealthy AZs within a given Region.

D. Reduce the TTL value for latency-based routing to detect changes more quickly. A lower TTL (Time To Live) value determines how long DNS resolvers cache a record. While a lower TTL helps changes (like removing an unhealthy record) propagate faster, it doesn't provide the detection mechanism itself. Without a health check, Route 53 wouldn't know an endpoint is unhealthy, regardless of the TTL.

Relevant Cloud Computing Concepts:

Route 53 Health Checks: A crucial feature for monitoring the health of specified endpoints and enabling automated DNS failover.

Weighted Routing Policy: Distributes traffic to multiple resources based on weights you assign, often used with health checks for intra-Region failover.

High Availability & Fault Tolerance: The ability of a system to operate continuously without failure for a long time, often achieved through redundancy and automatic failover mechanisms like health checks.

Availability Zones (AZs): Isolated locations within an AWS Region designed to be insulated from failures in other AZs, enabling resilient architectures when combined with proper health monitoring.

Authoritative Links for Further Research:

Configuring health checks for your domain and routing traffic to healthy Amazon EC2 instances:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/monitoring-health-of-endpoints.html>

Choosing a routing policy:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/AboutRoutingPolicies.html>

Question: 13

CertyIQ

A company must ensure that all Amazon EC2 Windows instances that are launched in an AWS account have a third-party agent installed. The company uses AWS Systems Manager, and the Windows instances are tagged appropriately. The company must deploy periodic updates to the third-party agent when the updates become available.

Which combination of steps will meet these requirements with the LEAST operational effort? (Choose two.)

- A. Create a Systems Manager Distributor package for the third-party agent.
- B. Create a Systems Manager OpsItem that includes the tag value for Windows. Attach the Systems Manager inventory to the OpsItem.
- C. Create an AWS Lambda function. Program the Lambda function to log in to each instance and to install or update the third-party agent as needed.
- D. Create a Systems Manager State Manager association to run the AWS-RunRemoteScript document Populate the details of the third-party agent package.
- E. Create a Systems Manager State Manager association to run the AWS-ConfigureAWSPackage document. Populate the details of the third-party agent package. Specify instance tags based on the appropriate tag value for Windows.

Answer: AE

Explanation:

The correct combination of steps to meet the requirements with the LEAST operational effort is A and E.

Detailed Justification:

1. A. Create a Systems Manager Distributor package for the third-party agent.

Purpose: AWS Systems Manager Distributor is specifically designed to package and distribute your own software or third-party agents to managed instances. This centralizes the installation files, scripts, and versioning for the third-party agent.

Operational Effort: This significantly reduces operational effort compared to manually scripting installations or maintaining separate repositories. Once created, the Distributor package provides a standardized and repeatable way to deploy the agent, handling different versions and making updates straightforward. It eliminates the need for complex custom logic to download and execute installers.

Relevance: This step is foundational for effectively managing a third-party agent within the AWS ecosystem using Systems Manager, preparing the agent for automated deployment.

Authoritative Link: AWS Systems Manager Distributor: <https://docs.aws.amazon.com/systems-manager/latest/userguide/distributor.html>

2. E. Create a Systems Manager State Manager association to run the AWS-ConfigureAWSPackage document. Populate the details of the third-party agent package. Specify instance tags based on the appropriate tag value for Windows.

Purpose: AWS Systems Manager State Manager allows you to define and maintain a desired configuration for your instances. The AWS-ConfigureAWSPackage document is a pre-built Systems Manager document specifically designed to install, uninstall, or update packages managed by Systems Manager Distributor.

Operational Effort: By creating a State Manager association, you automate the continuous deployment and updates of the third-party agent. State Manager will periodically check the compliance state of the instances and automatically apply the necessary actions (install or update the agent) based on the specified Distributor package. This eliminates manual intervention for installations and ensures periodic updates are applied without ongoing human oversight.

Targeting: Specifying instance tags ensures that the association targets only the relevant Amazon EC2 Windows instances, as required by the problem statement. This provides precise control and prevents unintended deployments.

Relevance: This step leverages the power of Systems Manager for continuous configuration management, ensuring the agent is always installed and up-to-date on all designated instances with minimal administrative burden.

Authoritative Link: AWS Systems Manager State Manager: <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-state.html>

Authoritative Link: AWS-ConfigureAWSPackage document: <https://docs.aws.amazon.com/systems-manager/latest/userguide/ssm-agent-documents-awsconfigureawspackage.html>

Why other options are less suitable:

B. Create a Systems Manager OpsItem...: OpsItems are for operational issues and tracking, not for software deployment and configuration management. This would not install or update the agent.

C. Create an AWS Lambda function...: While technically possible, programming a Lambda function to log into instances and manage installations/updates is a high-effort, custom solution. It would require significant development, credential management, error handling, and maintenance, which is not the "least operational effort."

D. Create a Systems Manager State Manager association to run the AWS-RunRemoteScript document...: AWS-RunRemoteScript would require you to write and maintain a custom script for installation and updates, which would need to handle download, execution, error handling, and idempotency. This is more complex and higher effort than using the purpose-built AWS-ConfigureAWSPackage document with a Distributor package.

By combining Distributor (A) to package the agent and State Manager with AWS-ConfigureAWSPackage (E) to deploy and update it, the company can achieve its goal with a highly automated and low-maintenance solution

Question: 14

A company has deployed Amazon EC2 instances from custom Amazon Machine Images (AMIs) in two AWS Regions. The company registered all the instances with AWS Systems Manager. The company discovers that the operating system on some instances has a significant zero-day exploit. However, the company does not know how many instances are affected. A CloudOps engineer must implement a solution to deploy operating system patches for the affected EC2 instances. Which solution will meet this requirement with the LEAST operational overhead?

- A. Define a patch baseline in Systems Manager Patch Manager. Use a Patch Manager scan to identify the affected instances. Use the Patch Now option in each Region to update the affected instances.
- B. Use AWS Config to identify the affected instances. Define a patch baseline in Systems Manager Patch Manager. Use the Patch Now option in Patch Manager to update the affected instances.
- C. Create an Amazon EventBridge rule to react to Systems Manager Compliance events. Configure the EventBridge rule to run a patch baseline on the affected instances.
- D. Use AWS Config to identify the affected instances. Update the existing EC2 AMIs with the desired patch. Manually launch instances from the new AMIs to replace the affected instances in both Regions.

Answer: A

Explanation:

The correct answer is A because it leverages AWS Systems Manager Patch Manager, which is purpose-built for identifying and applying operating system patches with the least operational overhead in this scenario.

Here's a detailed justification:

1. **Addressing Unknown Affected Instances:** The problem states the company "does not know how many instances are affected." Systems Manager Patch Manager's core functionality includes scanning instances to determine their patch compliance status against a defined patch baseline. This capability directly solves the problem of identifying exactly which instances are missing the critical zero-day patch without manual inspection or complex tooling.
2. **Leveraging Existing Infrastructure:** All instances are already registered with AWS Systems Manager. This means the necessary agent is installed, and instances are ready to be managed by Systems Manager services like Patch Manager, requiring no additional setup for connectivity or agent deployment.
3. **Efficient Patch Identification:** Defining a patch baseline allows the company to specify which patches are approved for deployment, including the one addressing the zero-day exploit. A Patch Manager scan operation will then compare the patches installed on each instance against this baseline, precisely pinpointing the non-compliant (affected) instances. This is far more direct and efficient for identifying missing patches than using AWS Config, which primarily tracks resource configuration and compliance against desired states rather than granular patch levels.
4. **Targeted and Immediate Remediation:** Once affected instances are identified through the scan, the "Patch Now" option in Systems Manager Patch Manager provides a straightforward way to deploy the required operating system patches immediately. This feature allows for on-demand patching of a specific set of instances or instances matching certain criteria, making it ideal for a critical, time-sensitive zero-day exploit. It avoids the overhead of setting up recurring maintenance windows if this is a one-off emergency patch.

5. Least Operational Overhead:

Compared to B (AWS Config for identification): While AWS Config can be used for compliance, identifying specific missing patches for a zero-day exploit using Config rules would be more complex to set up and maintain than Patch Manager's built-in scan functionality, which is designed for this exact purpose.

Compared to C (EventBridge automation): EventBridge rules with Compliance events are excellent for ongoing, automated remediation, but the initial step of identifying the unknown number of affected instances is not directly covered by just reacting to existing compliance events. For an immediate, unknown-scope incident, a direct scan and "Patch Now" is simpler than configuring complex event-driven automation for a potentially one-off emergency.

Compared to D (AMI update and instance replacement): Replacing instances by building new AMIs and launching new instances is a significantly higher-overhead operation. It involves potential downtime, re-registration with services, IP address changes, and careful orchestration, especially across two Regions and an unknown number of instances. In-place patching is much less disruptive and quicker for applying a single OS patch.

In summary, Option A utilizes the integrated capabilities of AWS Systems Manager Patch Manager for efficient identification of affected instances and immediate, targeted patching, directly addressing all requirements with minimal setup and execution complexity for an emergency patch deployment.

Authoritative Links:

AWS Systems Manager Patch Manager: <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-patch.html>

About Patch Baselines: <https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager-baselines.html>

Using Patch Now: <https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager-patch-now.html>

Question: 15

CertyIQ

A company hosts an FTP server on Amazon EC2 instances. In the company's AWS environment, AWS Security Hub sends findings for the EC2 instances to Amazon EventBridge because the FTP port has become publicly exposed in the security groups that are attached to the instances.

A CloudOps engineer wants an automated solution to remediate the Security Hub finding and any similar exposed port findings. The CloudOps engineer wants to use an event-driven approach.

Which solution will meet these requirements?

- A. Configure the existing EventBridge event to stop the EC2 instances that have the exposed port.
- B. Create a cron job for the FTP server to invoke an AWS Lambda function. Configure the Lambda function to modify the security group of the identified EC2 instances and to remove the instances that allow public access.
- C. Create a cron job for the FTP server that invokes an AWS Lambda function. Configure the Lambda function to modify the server to use SFTP instead of FTP.
- D. Configure the existing EventBridge event to invoke an AWS Lambda function. Configure the function to remove the security group rule that allows public access.

Answer: D

Explanation:

The most effective solution is to configure the existing Amazon EventBridge event to invoke an AWS Lambda function, which then removes the problematic security group rule. This approach directly addresses the

prompt's requirements for an automated, event-driven remediation of Security Hub findings for exposed ports. When AWS Security Hub detects a publicly exposed port on an EC2 instance, it publishes this finding to EventBridge.

EventBridge, acting as an event router, can then trigger a specific AWS Lambda function in response to this particular event pattern. The Lambda function, being serverless, executes code without requiring server provisioning or management, making it ideal for reactive automation. Inside the Lambda function, code written using the AWS SDK can programmatically identify the affected security group and the specific ingress rule allowing public access to the FTP port (typically port 21). It then calls the `revokeSecurityGroupIngress` API action to remove that rule.

This solution is superior because it is precisely targeted: it fixes the exact security misconfiguration without affecting the operational status of the EC2 instance or the FTP server itself, unless the server genuinely requires public access, in which case the finding would need a different resolution strategy. It aligns with the principle of least privilege in remediation, making the minimal necessary change to restore security compliance.

In contrast, option A, stopping the EC2 instances, is overly disruptive. While it would technically remove the public exposure, it causes unnecessary service downtime and impacts availability, which is generally not the first choice for a security group misconfiguration. Option B, using a cron job, deviates from the requested event-driven approach and, more critically, "removing the instances that allow public access" is an extreme measure that would lead to data loss and complete service disruption, far beyond merely fixing a security group rule. Option C also involves a cron job, failing the event-driven requirement, and changing the server to use SFTP is an application-level modification that requires careful planning and testing, not an immediate automated security group fix. Such an extensive change is outside the scope of remediating a network security group finding and could introduce new issues or break existing client connections. Therefore, configuring the EventBridge event to trigger a Lambda function that specifically removes the offending security group rule is the most efficient, targeted, and least disruptive automated solution.

Authoritative Links for Further Research:

AWS Security Hub: <https://aws.amazon.com/security-hub/>

Amazon EventBridge: <https://aws.amazon.com/eventbridge/>

AWS Lambda: <https://aws.amazon.com/lambda/>

Security Groups for your VPC: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Automated Remediation with AWS Security Hub, EventBridge, and Lambda:

<https://aws.amazon.com/blogs/security/how-to-implement-automated-security-response-and-remediation-with-aws-security-hub-and-aws-lambda/>

Question: 16

CertyIQ

A company plans to migrate several of its high performance computing (HPC) virtual machines (VMs) to Amazon EC2 instances on AWS. A CloudOps engineer must identify a placement group for this deployment. The strategy must minimize network latency and must maximize network throughput between the HPC VMs. Which strategy should the CloudOps engineer choose to meet these requirements?

- A. Deploy the instances in a cluster placement group in one Availability Zone.
- B. Deploy the instances in a partition placement group in two Availability Zones.
- C. Deploy the instances in a partition placement group in one Availability Zone.
- D. Deploy the instances in a spread placement group in two Availability Zones.

Answer: A

Explanation:

The optimal strategy to minimize network latency and maximize network throughput between high performance computing (HPC) virtual machines (VMs) on Amazon EC2 is to deploy them in a cluster placement group within a single Availability Zone.

A **cluster placement group** is specifically designed for applications that require low network latency and high network throughput, which are critical characteristics for tightly coupled HPC workloads. When instances are launched into a cluster placement group, Amazon EC2 attempts to place them as close together as possible within the same Availability Zone. This co-location on the same underlying hardware, or within the same network segment, dramatically reduces the network distance between instances, directly leading to lower latency and higher bandwidth communication.

Deploying instances in **one Availability Zone** is crucial because network traffic between different Availability Zones incurs additional latency due to physical separation and routing. HPC applications are often sensitive to even slight increases in inter-node communication time, making a single AZ deployment essential for achieving the best performance.

Let's look at why other options are unsuitable:

Partition placement groups are designed for large distributed and replicated workloads, such as HDFS or Apache Cassandra, that need to be resilient to hardware failures by spreading instances across different logical partitions (and thus different racks) within an Availability Zone or across multiple AZs. While they offer fault tolerance, their primary goal is not to minimize inter-instance latency to the same extent as cluster placement groups. Using two Availability Zones, as in option B, would further increase latency, directly contradicting the requirement.

Spread placement groups are designed to place instances on distinct underlying hardware racks, maximizing fault tolerance for a small number of critical instances. This strategy ensures that a failure of a single rack does not impact multiple instances. However, this separation inherently increases network distance and latency between instances compared to a cluster placement group, making it unsuitable for high-throughput, low-latency HPC needs. Deploying across two Availability Zones, as in option D, would again introduce unacceptable latency.

Therefore, choosing a cluster placement group in a single Availability Zone (Option A) is the only strategy that aligns perfectly with the requirements of minimizing network latency and maximizing network throughput for HPC applications.

Authoritative Links for further research:

Amazon EC2 Placement Groups: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/placement-groups.html>

AWS High Performance Computing (HPC): <https://aws.amazon.com/hpc/>

Question: 17

CertyIQ

A company manages a set of accounts on AWS by using AWS Organizations. The company's security team wants to use a native AWS service to regularly scan all AWS accounts against the Center for Internet Security (CIS) AWS Foundations Benchmark.

What is the MOST operationally efficient way to meet these requirements?

A. Designate a central security account as the AWS Security Hub administrator account. Create a script that sends an invitation from the Security Hub administrator account and accepts the invitation from the member account. Run the script every time a new account is created. Configure Security Hub to run the CIS AWS Foundations Benchmark scans.

B. Run the CIS AWS Foundations Benchmark across all accounts by using Amazon Inspector.

C. Designate a central security account as the Amazon GuardDuty administrator account. Create a script that sends an invitation from the GuardDuty administrator account and accepts the invitation from the member account. Run the script every time a new account is created. Configure GuardDuty to run the CIS AWS Foundations Benchmark scans.

D. Designate an AWS Security Hub administrator account. Configure new accounts in the organization to automatically become member accounts. Enable CIS AWS Foundations Benchmark scans.

Answer: D

Explanation:

The most operationally efficient way to meet these requirements is by leveraging AWS Security Hub's native integration with AWS Organizations. AWS Security Hub is specifically designed for security posture management, which includes conducting automated security checks against industry standards like the Center for Internet Security (CIS) AWS Foundations Benchmark.

First, designating a central security account as the AWS Security Hub administrator account allows for centralized management and monitoring of security posture across all accounts in the organization. This provides a single pane of glass for security findings.

Crucially, Security Hub's integration with AWS Organizations enables a powerful feature: the administrator account can configure Security Hub to automatically enable itself and enroll new accounts created within the organization as member accounts. This eliminates the need for manual invitations, acceptance scripts, or per-account configuration, which significantly reduces operational overhead and improves scalability as the organization grows.

Once Security Hub is enabled and accounts are enrolled, the administrator can then configure it to run the CIS AWS Foundations Benchmark scans. Security Hub natively supports this standard, continuously monitoring the security configurations of all member accounts against the benchmark's controls. This ensures ongoing compliance without manual intervention.

In contrast, options A and C propose creating and running scripts to send and accept invitations for every new account. This approach is far less operationally efficient than Security Hub's automatic enablement feature for organizations. Options B and C also incorrectly suggest using Amazon Inspector or Amazon GuardDuty for running the CIS AWS Foundations Benchmark scans; Amazon Inspector focuses on vulnerability management for EC2 and containers, and GuardDuty is a threat detection service, neither of which is designed for comprehensive configuration compliance auditing against benchmarks like CIS AWS Foundations Benchmark. Therefore, leveraging Security Hub's organizational capabilities for automated enrollment and its native support for the CIS benchmark makes option D the optimal and most efficient solution.

Authoritative Links:

AWS Security Hub with AWS Organizations:

<https://docs.aws.amazon.com/securityhub/latest/userguide/securityhub-orgs-integration.html>

CIS AWS Foundations Benchmark in Security Hub:

<https://docs.aws.amazon.com/securityhub/latest/userguide/standards-cis.html>

Question: 18

CertyIQ

A company has users that deploy Amazon EC2 instances that have more volume performance capacity than is required. A CloudOps engineer needs to review all Amazon Elastic Block Store (Amazon EBS) volumes that are associated with the instances and create cost optimization recommendations based on IOPS and throughput. What should the CloudOps engineer do to meet these requirements in the MOST operationally efficient way?

- A. Use the monitoring graphs in the EC2 console to view metrics for EBS volumes. Review the consumed space against the provisioned space on each volume. Identify any volumes that have low utilization.
- B. Stop the EC2 instances from the EC2 console. Change the EC2 instance type to Amazon EBS-optimized. Start the EC2 instances.
- C. Opt in to AWS Compute Optimizer. Allow sufficient time for metrics to be gathered. Review the Compute Optimizer findings for EBS volumes.
- D. Install the fio tool onto the EC2 instances and create a .cfg file to approximate the required workloads. Use the benchmark results to gauge whether the provisioned EBS volumes are of the most appropriate type.

Answer: C

Explanation:

The most operationally efficient way to review Amazon EBS volumes for cost optimization based on IOPS and throughput is to opt in to AWS Compute Optimizer.

AWS Compute Optimizer is a fully managed service that analyzes historical utilization metrics from Amazon CloudWatch for various AWS resources, including EBS volumes. For EBS volumes, it specifically evaluates metrics like IOPS (I/O operations per second), throughput, and queue depth over time. Based on this analysis and using machine learning, Compute Optimizer provides intelligent, data-driven recommendations for optimal EBS volume configurations, such as suggesting a change to a smaller volume type (e.g., transitioning from gp2 to gp3) or a different performance tier that aligns better with actual workload demands.

This approach is highly operationally efficient because it automates the complex process of data collection, analysis, and recommendation generation across potentially hundreds or thousands of EBS volumes. CloudOps engineers do not need to manually sift through individual CloudWatch graphs (as in option A), which would be time-consuming and prone to human error, especially at scale. Compute Optimizer directly addresses the requirement for cost optimization based on IOPS and throughput by identifying volumes that are provisioned with more capacity than they consistently utilize.

In contrast, option A, using manual monitoring graphs, is not scalable or efficient for a comprehensive review across many volumes. Option B, changing instance types, is irrelevant to optimizing the EBS volumes themselves and is disruptive. Option D, installing and configuring the fio tool, involves significant manual effort, requires specialized expertise, consumes compute resources, and only provides benchmark results for a simulated workload, rather than insights from actual, sustained usage patterns across an entire fleet. Compute Optimizer provides continuous, intelligent insights without requiring any manual testing or disruptive changes upfront, making it the superior choice for operational efficiency.

Authoritative Links:

AWS Compute Optimizer documentation: <https://aws.amazon.com/compute-optimizer/>

Optimizing Amazon EBS volumes with AWS Compute Optimizer: <https://docs.aws.amazon.com/compute-optimizer/latest/ug/ebs-volumes-recommendations.html>

Question: 19

CertyIQ

A CloudOps engineer must ensure that all of a company's current and future Amazon S3 buckets have logging enabled. If an S3 bucket does not have logging enabled, an automated process must enable logging for the S3 bucket.

Which solution will meet these requirements?

- A. Use AWS Trusted Advisor to perform a check for S3 buckets that do not have logging enabled. Configure the check to enable logging for S3 buckets that do not have logging enabled.
- B. Configure an S3 bucket policy that requires all current and future S3 buckets to have logging enabled.

- C. Use the s3-bucket-logging-enabled AWS Config managed rule. Add a remediation action that uses an AWS Lambda function to enable logging.
- D. Use the s3-bucket-logging-enabled AWS Config managed rule. Add a remediation action that uses the AWS-ConfigureS3BucketLogging AWS Systems Manager Automation runbook to enable logging.

Answer: D

Explanation:

The most effective solution for ensuring all Amazon S3 buckets have logging enabled and automatically enabling it if not is **D. Use the s3-bucket-logging-enabled AWS Config managed rule. Add a remediation action that uses the AWS-ConfigureS3BucketLogging AWS Systems Manager Automation runbook to enable logging.**

Here's a detailed justification:

1. **Continuous Compliance Monitoring with AWS Config:** The core requirement is to continuously monitor and enforce logging across current and future S3 buckets. AWS Config is purpose-built for this. It provides a detailed inventory of AWS resources, tracks configuration changes, and evaluates these configurations against desired state rules.
2. **Specific Managed Rule for S3 Logging:** The s3-bucket-logging-enabled AWS Config managed rule directly addresses the need to check if S3 buckets have logging enabled. Managed rules are pre-defined by AWS, making them easy to deploy and maintain without writing custom logic.
3. **Automated Remediation:** Once AWS Config identifies a non-compliant S3 bucket (i.e., one without logging enabled), it needs an automated mechanism to rectify the situation. AWS Config allows associating remediation actions with non-compliant rules.
4. **AWS Systems Manager Automation Runbook for Remediation:** Option D leverages AWS Systems Manager Automation runbooks. Specifically, the AWS-ConfigureS3BucketLogging runbook is a pre-built, managed Automation document provided by AWS designed precisely for this task: enabling S3 bucket logging.
5. **Advantages of Managed Runbooks:** Using a managed runbook like AWS-ConfigureS3BucketLogging offers significant advantages over custom solutions (like a Lambda function in option C). It reduces operational overhead because AWS maintains and updates the runbook, ensuring its reliability and security. There's no custom code to write, test, or debug, which minimizes development effort and potential for errors.
6. **Why other options are incorrect:**

A. AWS Trusted Advisor: While Trusted Advisor can identify security misconfigurations like S3 buckets without logging, it is primarily a reporting and advisory tool. It does not natively provide automated remediation capabilities to enable logging for non-compliant resources.

B. S3 bucket policy: An S3 bucket policy defines permissions for accessing the bucket and its objects. It cannot be used to enable logging on the bucket itself or to enforce logging across all buckets within an account automatically. Bucket policies are for access control, not configuration enforcement or automated provisioning.

C. AWS Config + Lambda function: This is a viable technical solution, as AWS Config can trigger a custom Lambda function for remediation. However, option D is superior because it utilizes a managed AWS Systems Manager Automation runbook (AWS-ConfigureS3BucketLogging) for the remediation action. This eliminates the need to develop, test, and maintain custom Lambda code, making option D more efficient, reliable, and operationally sound for a common task.

In summary, AWS Config provides the continuous monitoring and evaluation, and the s3-bucket-logging-enabled rule specifically targets the requirement. The integration with AWS Systems Manager Automation and the pre-built AWS-ConfigureS3BucketLogging runbook offers a robust, low-maintenance, and automated solution for compliance enforcement.

Authoritative Links for Further Research:

AWS Config: <https://aws.amazon.com/config/>

s3-bucket-logging-enabled AWS Config Managed Rule:

<https://docs.aws.amazon.com/config/latest/developerguide/s3-bucket-logging-enabled.html>

AWS Systems Manager Automation: <https://aws.amazon.com/systems-manager/features/automation/>

AWS-ConfigureS3BucketLogging Automation Document: You can find details on this runbook within the AWS Systems Manager console or documentation related to Automation runbooks.

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-automation-runbooks.html> (Search for the specific runbook name within the console's Automation documents for parameters and details.)

Remediating Noncompliant AWS Resources with AWS Config:

<https://docs.aws.amazon.com/config/latest/developerguide/auto-remediation.html>

Question: 20

CertyIQ

A company has millions of subscribers. The company's marketing department wants to automate a process that sends notifications to subscribers every Saturday. The company already has a mechanism that uses Amazon Simple Notification Service (Amazon SNS) to send notifications to subscribers. However, the company has historically sent notifications to subscribers manually.

A CloudOps engineer needs a solution to automatically send notifications on a schedule.

Which solution will meet these requirements in the MOST operationally efficient way?

- A. Launch a new Amazon EC2 instance. Configure a cron job to use the AWS SDK to send an SNS notification to subscribers every Saturday.
- B. Create a rule in Amazon EventBridge that triggers every Saturday. Configure the rule to publish a notification to an SNS topic.
- C. Create an SNS subscription to a message fanout that sends notifications to subscribers every Saturday.
- D. Use the AWS Step Functions scheduling feature to run a Step Functions step every Saturday. Configure the step to publish a message to an SNS topic.

Answer: B

Explanation:

The most operationally efficient solution is **B. Create a rule in Amazon EventBridge that triggers every Saturday. Configure the rule to publish a notification to an SNS topic.**

Here's why:

Amazon EventBridge is a serverless event bus that makes it easy to connect applications together using data from your own applications, integrated SaaS applications, and AWS services. For scheduling tasks, EventBridge (formerly CloudWatch Events) provides a robust and managed cron-like capability. By creating an EventBridge rule with a schedule expression (e.g., `cron(0 10 ? SAT)` for 10 AM UTC every Saturday), you can directly invoke an AWS service or target. In this scenario, the target would be the existing Amazon SNS topic.

This approach offers significant operational efficiency because it is fully serverless. There are no servers, containers, or underlying infrastructure to provision, patch, or manage. EventBridge handles all the scalability, reliability, and availability concerns automatically. The cost is based on the number of events processed, which would be minimal for a weekly notification, making it highly cost-effective. The integration between EventBridge and SNS is direct and seamless, requiring minimal configuration. This "set it and forget it" model drastically reduces the operational overhead for the CloudOps engineer.

In contrast, option A, launching an EC2 instance and configuring a cron job, introduces substantial operational burden. The EC2 instance requires continuous management, including OS patching, security updates, monitoring, ensuring high availability, and managing its lifecycle. This is overkill for a simple weekly scheduled task and would incur higher costs for continuous running, even if the job itself runs briefly once a week.

Option C, "Create an SNS subscription to a message fanout that sends notifications to subscribers every Saturday," is fundamentally incorrect. Amazon SNS is a messaging service that facilitates fanout, but it does not have an inherent scheduling feature. An SNS subscription receives messages; it doesn't send messages on a schedule. You still need an external service to trigger messages to the SNS topic itself.

Option D, using AWS Step Functions scheduling, is technically feasible but less operationally efficient for this specific, simple use case. Step Functions is ideal for orchestrating complex, multi-step workflows, handling retries, parallel execution, and state management. For a single action like publishing directly to an SNS topic, introducing a Step Functions state machine adds an unnecessary layer of abstraction and complexity compared to EventBridge's direct target integration. While EventBridge can trigger Step Functions, for a simple direct publish, EventBridge directly targeting SNS is more streamlined and cost-effective.

Therefore, EventBridge provides the most straightforward, serverless, and operationally efficient solution for automatically sending SNS notifications on a schedule.

Authoritative Links for Further Research:

Amazon EventBridge User Guide: <https://docs.aws.amazon.com/eventbridge/latest/userguide/what-is-eventbridge.html>

Scheduling with Amazon EventBridge: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-create-rule-schedule.html>

Amazon SNS Developer Guide: <https://docs.aws.amazon.com/sns/latest/dg/welcome.html>

Question: 21

CertyIQ

A CloudOps engineer is responsible for a company's disaster recovery procedures. The company has a source Amazon S3 bucket in a production account, and it wants to replicate objects from the source to a destination S3 bucket in a nonproduction account. The CloudOps engineer configures S3 cross-Region, cross-account replication to copy the source S3 bucket to the destination S3 bucket. When the CloudOps engineer attempts to access objects in the destination S3 bucket, they receive an Access Denied error. Which solution will resolve this problem?

- A. Modify the replication configuration to change object ownership to the destination S3 bucket owner.
- B. Ensure that the replication rule applies to all objects in the source S3 bucket and is not scoped to a single prefix.
- C. Retry the request when the S3 Replication Time Control (S3 RTC) has elapsed.
- D. Verify that the storage class for the replicated objects did not change between the source S3 bucket and the destination S3 bucket.

Answer: A

Explanation:

The "Access Denied" error in a cross-account S3 replication scenario, specifically when trying to access objects in the destination bucket from the destination account, is a classic indication of an object ownership and permissions mismatch. By default, when objects are replicated from a source S3 bucket in one AWS account to a destination S3 bucket in another account, the objects retain their original owner, which is the source account. This means that even if the destination bucket has a policy allowing principals from its own

account to access objects, those permissions may be overridden or ineffective because the destination account does not own the objects.

To resolve this, the replication configuration must be explicitly modified to transfer object ownership to the destination S3 bucket owner. Amazon S3 provides mechanisms within the replication configuration to achieve this. One common method is to use the `OwnershipOverride` element in the replication rule, setting the owner to `DestinationBucketOwner`. Alternatively, if the S3 Object Ownership setting on the destination bucket is configured as "Bucket owner preferred" (which grants the destination bucket owner full control via ACLs) or "Bucket owner enforced" (which disables ACLs and makes the bucket owner the object owner), then replicated objects will be owned by the destination account. Once the destination account owns the replicated objects, its IAM users, roles, and bucket policies can effectively grant access to these objects, thereby resolving the "Access Denied" error.

Options B, C, and D do not address the fundamental ownership and access control issue:

B. Ensure that the replication rule applies to all objects: This affects which objects are replicated, not the access permissions for objects that are successfully replicated. If an object isn't replicated, it wouldn't be present to cause an "Access Denied" error.

C. Retry the request when S3 Replication Time Control (S3 RTC) has elapsed: S3 RTC guarantees replication latency but does not influence access permissions once an object has arrived in the destination bucket. If the object is present but inaccessible, waiting longer won't change its ownership or ACLs.

D. Verify that the storage class did not change: Changes in storage class impact cost and retrieval times but do not directly cause an "Access Denied" error based on permissions. Regardless of the storage class, the ownership issue would persist.

Therefore, modifying the replication configuration to ensure the destination S3 bucket owner takes ownership of the replicated objects is the correct solution.

Authoritative Links for Further Research:

Replicating objects when source and destination buckets are owned by different AWS accounts:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication-walkthrough-3.html>

Controlling object ownership and managing ACLs for your buckets:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/about-object-ownership.html>

Granting permissions when objects are created by other AWS accounts (Cross-account replication ownership override): <https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication-config-for-same-account.html#replication-config-same-account-create-other-account>

Question: 22

CertyIQ

An Amazon EC2 instance is running an application that uses Amazon Simple Queue Service (Amazon SQS) queues. A CloudOps engineer must ensure that the application can read write, and delete messages from the SQS queues. Which solution will meet these requirements in the MOST secure manner?

A. Create an IAM user with an IAM policy that allows the `sqs:SendMessage` permission, the `sqs:ReceiveMessage` permission, and the `sqs:DeleteMessage` permission to the appropriate queues. Embed the IAM user's credentials in the application's configuration.

B. Create an IAM user with an IAM policy that allows the `sqs:SendMessage` permission, the `sqs:ReceiveMessage` permission, and the `sqs:DeleteMessage` permission to the appropriate queues. Export the IAM user's access key and secret access key as environment variables on the EC2 instance.

C. Create and associate an IAM role that allows EC2 instances to call AWS services. Attach an IAM policy to the role that allows `sqs:*` permissions to the appropriate queues.

D. Create and associate an IAM role that allows EC2 instances to call AWS services. Attach an IAM policy to the role that allows the `sqs:SendMessage` permission, the `sqs:ReceiveMessage` permission, and the

sqs:DeleteMessage permission to the appropriate queues.

Answer: D

Explanation:

Option D is the most secure solution because it leverages IAM roles for EC2 instances, which is the recommended best practice for granting permissions to applications running on EC2.

Here's a detailed justification:

1. **IAM Roles for EC2 Instances (Security Best Practice):** When an IAM role is associated with an EC2 instance, the instance does not need to store any long-term AWS access keys or secret access keys directly. Instead, the instance can retrieve temporary security credentials from the EC2 instance metadata service. These temporary credentials are automatically rotated by AWS, significantly reducing the risk associated with compromised static credentials.
2. **Elimination of Hardcoded/Exposed Credentials:** Options A and B involve creating an IAM user and embedding or exporting its long-lived credentials directly onto the EC2 instance. This is a major security vulnerability. If the instance, application code, or configuration files are compromised, an attacker gains immediate access to these static credentials, potentially leading to unauthorized access to AWS resources. This violates the principle of "never hardcode credentials."
3. **Principle of Least Privilege (Granular Permissions):** Option D explicitly defines the necessary permissions: `sqs:SendMessage`, `sqs:ReceiveMessage`, and `sqs:DeleteMessage`. This adheres to the principle of least privilege, meaning the application is granted only the permissions it absolutely needs to perform its function and nothing more.
4. **Limitation of Blast Radius:** By granting only the required permissions, if the EC2 instance were compromised, an attacker's ability to interact with SQS (or other AWS services) would be severely limited to only sending, receiving, and deleting messages. They would not be able to create new queues, purge existing queues, or perform other administrative actions.
5. **Comparison with Option C:** While Option C also uses an IAM role, it grants `sqs:*` permissions. This is overly permissive as it allows all SQS actions (e.g., `CreateQueue`, `PurgeQueue`, `ListQueues`, etc.), not just the required message operations. This violates the principle of least privilege and would provide a broader attack surface if the instance were compromised.

In summary, Option D combines the most secure method for credential management (IAM roles with temporary, automatically rotated credentials) with the principle of least privilege (granting only necessary, granular permissions), making it the most secure solution.

Authoritative Links for Further Research:

Using an IAM role to grant permissions to applications running on Amazon EC2 instances:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html>

Security best practices in IAM: <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>

Principle of Least Privilege: <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/security.html#identify-and-control-who-can-do-what-with-which-resource-sec03>

Question: 23

CertyIQ

A CloudOps engineer is designing a solution for an Amazon RDS for PostgreSQL DB instance. Database credentials must be stored and rotated monthly. The applications that connect to the DB instance send write-intensive traffic with variable client connections that sometimes increase significantly in a short period of time.

Which solution should a CloudOps engineer choose to meet these requirements?

- A. Configure AWS Key Management Service (AWS KMS) to automatically rotate the keys for the DB instance. Use RDS Proxy to handle the increases in database connections.
- B. Configure AWS Key Management Service (AWS KMS) to automatically rotate the keys for the DB instance. Use RDS read replicas to handle the increases in database connections.
- C. Configure AWS Secrets Manager to automatically rotate the credentials for the DB instance. Use RDS Proxy to handle the increases in database connections.
- D. Configure AWS Secrets Manager to automatically rotate the credentials for the DB instance. Use RDS read replicas to handle the increases in database connections.

Answer: C

Explanation:

The correct solution is C because it directly addresses both key requirements effectively.

First, for storing and rotating database credentials monthly, **AWS Secrets Manager** is the appropriate service. It is specifically designed to manage, store, and automatically rotate sensitive information like database credentials. Secrets Manager offers native integration with Amazon RDS for PostgreSQL, enabling automatic rotation of the database user's password without requiring changes to the application code. This meets the crucial security and compliance requirement for regular credential rotation. In contrast, AWS Key Management Service (AWS KMS) is used for managing encryption keys, not for the automatic rotation of database usernames and passwords themselves, although Secrets Manager uses KMS to encrypt the secrets it stores.

Second, for handling write-intensive traffic with variable and potentially significant increases in client connections, **Amazon RDS Proxy** is the ideal choice. RDS Proxy provides connection pooling, which reuses established database connections. This significantly reduces the overhead on the database instance from opening and closing new connections, which is particularly beneficial during sudden connection spikes. It also multiplexes application connections to a smaller number of database connections, preventing the database from being overwhelmed and improving its performance and availability. Since the traffic is write-intensive, the primary DB instance needs protection and efficient connection handling, which RDS Proxy delivers by sitting between the applications and the database.

Using RDS read replicas (as suggested in options B and D) would be suitable for offloading read-intensive traffic, thereby improving read scalability. However, read replicas do not handle write operations; all writes must still go to the primary database instance. Therefore, read replicas would not solve the problem of managing connections or mitigating connection spikes for the write-intensive workload on the primary instance.

In summary, AWS Secrets Manager provides the secure and automated credential rotation, while RDS Proxy ensures efficient connection management and resilience for the write-intensive database with fluctuating connection demands. This combination aligns perfectly with all the stated requirements.

Authoritative Links:

AWS Secrets Manager: <https://aws.amazon.com/secrets-manager/>

AWS Secrets Manager for RDS Rotation:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/reference_available-rotation-templates.html#templatedetails_rds_singleuser

Amazon RDS Proxy: <https://aws.amazon.com/rds/proxy/>

Amazon RDS Proxy Benefits: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html#rds-proxy-benefits>

Question: 24

A company operates compute resources in a VPC and in the company's on-premises data center. The company already has an AWS Direct Connect connection between the VPC and the on-premises data center. A CloudOps engineer needs to ensure that Amazon EC2 instances in the VPC can resolve DNS names for hosts in the on-premises data center.

Which solution will meet this requirement with the LEAST amount of ongoing maintenance?

- A. Create an Amazon Route 53 private hosted zone. Populate the zone with the hostnames and IP addresses of the hosts in the on-premises data center.
- B. Create an Amazon Route 53 Resolver outbound endpoint. Add the IP addresses of an on-premises DNS server for the domain names that need to be forwarded.
- C. Set up a forwarding rule for reverse DNS queries in Amazon Route 53 Resolver. Set the `enableDnsHostnames` attribute to true for the VPC.
- D. Add the hostnames and IP addresses for the on-premises hosts to the `/etc/hosts` file of each EC2 instance.

Answer: B

Explanation:

The correct solution is B because it provides the most automated and low-maintenance approach for hybrid DNS resolution.

Amazon Route 53 Resolver outbound endpoints enable Amazon EC2 instances in a Virtual Private Cloud (VPC) to resolve DNS names for hosts located in an on-premises data center. This is achieved by configuring forwarding rules within Route 53 Resolver. When an EC2 instance attempts to resolve a DNS name for a domain specified in a forwarding rule (e.g., `onprem.example.com`), the query is directed through the outbound endpoint to the designated on-premises DNS servers.

The "least amount of ongoing maintenance" is a critical aspect. Once the outbound endpoint and its associated forwarding rules are configured, any changes to the hostnames or IP addresses of on-premises resources are managed solely within the on-premises DNS infrastructure. There is no need to update AWS configurations for these on-premises changes. New EC2 instances launched in the VPC will automatically leverage the VPC's default DNS resolver, which will then use the outbound endpoint for on-premises name resolution without any per-instance configuration. This makes it a highly scalable and maintenance-efficient solution for hybrid environments.

Let's evaluate why other options are less suitable:

A. Create an Amazon Route 53 private hosted zone. Populate the zone with the hostnames and IP addresses of the hosts in the on-premises data center. This approach would require manual duplication and synchronization of all on-premises DNS records into the Route 53 private hosted zone. Every time an on-premises host is added, removed, or its IP address changes, the Route 53 private hosted zone would need a manual update. This is extremely high maintenance and error-prone.

C. Set up a forwarding rule for reverse DNS queries in Amazon Route 53 Resolver. Set the `enableDnsHostnames` attribute to true for the VPC. This option addresses reverse DNS (IP to name) or AWS-assigned hostnames, not the core problem of EC2 instances resolving forward DNS (name to IP) for on-premises hosts. While `enableDnsHostnames` is good practice for AWS-managed hostnames, it does not facilitate resolution of on-premises names.

D. Add the hostnames and IP addresses for the on-premises hosts to the `/etc/hosts` file of each EC2 instance. This is a highly unscalable and labor-intensive solution. Each EC2 instance would need to have its `/etc/hosts` file individually updated. Any change in the on-premises environment (e.g., adding a new server, changing an IP) would necessitate updates across all relevant EC2 instances. This introduces significant

operational overhead and is completely antithetical to the "least ongoing maintenance" requirement.

Therefore, Route 53 Resolver outbound endpoints offer a centralized, automated, and low-maintenance way to enable DNS resolution between VPC resources and on-premises data centers, leveraging the existing Direct Connect connection.

Authoritative Links for Further Research:

What is Amazon Route 53 Resolver?

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

Forwarding outbound DNS queries to your network

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-outbound-setup.html>

Creating a Route 53 Resolver endpoint

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-endpoints-operations.html>

Question: 25

CertyIQ

A company hosts a web application on an Amazon EC2 instance. The web server logs are published to Amazon CloudWatch Logs. The log events have the same structure and include the HTTP response codes that are associated with the user requests. The company needs to monitor the number of times that the web server returns an HTTP 404 response.

What is the MOST operationally efficient solution that meets these requirements?

- A. Create a CloudWatch Logs metric filter that counts the number of times that the web server returns an HTTP 404 response.
- B. Create a CloudWatch Logs subscription filter that counts the number of times that the web server returns an HTTP 404 response.
- C. Create an AWS Lambda function that runs a CloudWatch Logs Insights query that counts the number of 404 codes in the log events during the past hour.
- D. Create a script that runs a CloudWatch Logs Insights query that counts the number of 404 codes in the log events during the past hour.

Answer: A

Explanation:

The most operationally efficient solution is to create a CloudWatch Logs metric filter. This method directly addresses the requirement of monitoring HTTP 404 responses with minimal overhead and real-time processing.

A CloudWatch Logs metric filter defines a pattern that CloudWatch Logs uses to extract metrics from incoming log events. When a log event containing the specified pattern (e.g., "HTTP 404") arrives, the filter increments a custom CloudWatch metric. This makes it a push-based, real-time mechanism, processing logs as they are ingested without requiring any additional compute resources. The resulting custom metric can then be used to create CloudWatch alarms for immediate notification when 404s exceed a certain threshold. This approach is highly efficient because it eliminates the need for separate applications or services to analyze log data and publish metrics, significantly reducing operational complexity and cost.

In contrast, a CloudWatch Logs subscription filter (Option B) merely routes log events to another destination like AWS Lambda, Kinesis Data Streams, or Kinesis Firehose. While it can be a part of a solution, it doesn't inherently count the 404 responses. You would still need to develop and manage an additional component

(e.g., a Lambda function) to process the forwarded logs, extract the 404 codes, and then publish a custom metric to CloudWatch, adding complexity, latency, and cost.

Options C and D, which involve running CloudWatch Logs Insights queries, are less efficient for continuous, real-time monitoring. CloudWatch Logs Insights is excellent for interactive, ad-hoc analysis and troubleshooting historical data, but running it periodically (e.g., hourly as specified) via a Lambda function or a script introduces significant delays in detecting issues. An hourly query means a 404 spike might not be noticed for up to an hour, which is unacceptable for critical web application monitoring. Furthermore, managing the Lambda function or script, along with paying for its execution and the Logs Insights query costs, adds operational burden and can be more expensive than a simple metric filter for this specific use case.

Therefore, creating a CloudWatch Logs metric filter (Option A) is the most operationally efficient choice as it provides real-time, automated monitoring with the lowest operational overhead and cost for detecting specific patterns in log streams.

Authoritative Links for further research:

CloudWatch Logs Metric Filters:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/FilterAndPatternSyntax.html>

CloudWatch Logs Subscription Filters:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/SubscriptionFilters.html>

CloudWatch Logs Insights: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/CloudWatch-Logs-Insights-tutorial.html>

Question: 26

CertyIQ

A company has an internal web application that runs on Amazon EC2 instances behind an Application Load Balancer. The instances run in an Amazon EC2 Auto Scaling group in a single Availability Zone. A CloudOps engineer must make the application highly available.

Which action should the CloudOps engineer take to meet this requirement?

- A. Increase the maximum number of instances in the Auto Scaling group to meet the capacity that is required at peak usage.
- B. Increase the minimum number of instances in the Auto Scaling group to meet the capacity that is required at peak usage.
- C. Update the Auto Scaling group to launch new instances in a second Availability Zone in the same AWS Region.
- D. Update the Auto Scaling group to launch new instances in an Availability Zone in a second AWS Region.

Answer: C

Explanation:

The current setup, with all Amazon EC2 instances running in a single Availability Zone (AZ), represents a significant single point of failure for the web application. If that specific AZ experiences an outage, whether due to power failure, network issues, or other disruptions, the entire application would become inaccessible, rendering it unavailable.

High availability in AWS is primarily achieved by distributing application components across multiple, physically distinct, and independent Availability Zones within the same AWS Region. Each AZ is designed to be isolated from failures in other AZs, possessing its own power, networking, and cooling infrastructure.

Option C, "Update the Auto Scaling group to launch new instances in a second Availability Zone in the same

AWS Region," directly addresses this critical vulnerability. By configuring the Auto Scaling group to span multiple AZs, instances are automatically launched and maintained across these separate fault domains. Should one AZ become unhealthy or completely unavailable, the Application Load Balancer (ALB) will automatically detect the failure and stop routing traffic to the impaired instances in that AZ. Instead, it will seamlessly direct all incoming traffic to the healthy instances running in the remaining operational AZs, ensuring continuous application availability without manual intervention. This design is a fundamental AWS best practice for building fault-tolerant and highly available applications.

In contrast, options A and B, which suggest increasing the maximum or minimum number of instances, only address scalability and baseline capacity within the existing single AZ. While important for performance and handling traffic spikes, these actions do not protect against an AZ-wide outage; if the single AZ fails, all instances within it will fail regardless of their count.

Option D, "Update the Auto Scaling group to launch new instances in an Availability Zone in a second AWS Region," aims for multi-region disaster recovery, which provides resilience against an entire AWS Region outage. While offering an even higher level of resilience, the question asks to make the application "highly available." Distributing resources across multiple AZs within a single region is the primary and most direct method to achieve high availability against common AZ-level failures, and it's simpler and more cost-effective than a full multi-region deployment. Furthermore, Auto Scaling Groups are typically configured to operate within a single AWS Region, spanning its AZs, rather than directly launching instances across different regions as part of one ASG.

Authoritative Links:

AWS Global Infrastructure - Regions and Availability Zones: Provides foundational understanding of AZs and their role in high availability. https://aws.amazon.com/about-aws/global-infrastructure/regions_az/

AWS EC2 Auto Scaling - Health Checks and Availability Zones: Explains how Auto Scaling groups work with AZs to maintain application availability. <https://docs.aws.amazon.com/autoscaling/ec2/userguide/asg-health-check-AZs.html>

AWS Well-Architected Framework - Reliability Pillar: Highlights multi-AZ design as a core principle for reliability and high availability. <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/reliability-pillar.html>

Question: 27

CertyIQ

A CloudOps engineer is creating a simple, public-facing website running on Amazon EC2. The CloudOps engineer created the EC2 instance in an existing public subnet and assigned an Elastic IP address to the instance. Next, the CloudOps engineer created and applied a new security group to the instance to allow incoming HTTP traffic from 0.0.0.0/0. Finally, the CloudOps engineer created a new network ACL and applied it to the subnet to allow incoming HTTP traffic from 0.0.0.0/0. However, the website cannot be reached from the internet.

What is the cause of this issue?

- A. The CloudOps engineer did not create an outbound rule that allows ephemeral port return traffic in the new network ACL.
- B. The CloudOps engineer did not create an outbound rule in the security group that allows HTTP traffic from port 80.
- C. The Elastic IP address assigned to the EC2 instance has changed.
- D. There is an additional network ACL associated with the subnet that includes a rule that denies inbound HTTP traffic from port 80.

Answer: A

Explanation:

The issue lies with the stateless nature of Network Access Control Lists (NACLs) compared to the stateful nature of Security Groups. When a client initiates an HTTP connection to a web server (on port 80), the request arrives at the server from an ephemeral port on the client's side. The server then sends its response back from its well-known port (80) to the client's ephemeral port.

Security Groups are stateful, meaning if an inbound rule is created to allow traffic on port 80, the corresponding outbound return traffic (from port 80 to the client's ephemeral port) is automatically allowed without needing a separate outbound rule.

However, Network ACLs are stateless. This implies that for any traffic to flow through a NACL, both the inbound and outbound directions must be explicitly permitted. In this scenario, the CloudOps engineer correctly created an inbound rule in the NACL to allow HTTP traffic on port 80. Yet, for the website's responses to reach the internet, an explicit outbound rule is also required in the NACL. This outbound rule must allow traffic destined for the internet on ephemeral ports (typically 1024-65535, though often a broader range like 0-65535 is used for simplicity) from the EC2 instance's port 80. Without this outbound rule, the web server's responses are blocked at the subnet level by the NACL, even though the initial request was allowed to enter.

Therefore, the cause of the issue is that the CloudOps engineer did not create an outbound rule in the new network ACL that allows return traffic on ephemeral ports, which is essential for the HTTP responses to leave the subnet and reach the client.

AWS Documentation on Security Groups:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

AWS Documentation on Network ACLs: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

Question: 28

CertyIQ

A company wants to use AWS Systems Manager to manage a large fleet of Amazon EC2 instances. The company hosts the instances in private subnets. The company follows the principle of least privilege to assign access permissions. All private subnets have internet connectivity through a NAT gateway.

A CloudOps engineer installs the latest version of the Systems Manager Agent (SSM Agent). However, the EC2 instances do not appear in Systems Manager Fleet Manager. The CloudOps engineer must resolve this issue. Which solution will meet this requirement?

- A. Replace the NAT gateway with a NAT instance that is deployed in the public subnet. Update the private subnet's route table to use the NAT instance.
- B. Create a VPC endpoint for Systems Manager. Remove routes to the internet through the NAT gateway from the private subnet's route table.
- C. Attach the `AmazonSSMManagedInstanceCore` AWS managed policy to the EC2 instance profile that is associated with the instances.
- D. Attach a custom policy that allows all actions to `ssm*` to the EC2 instance profile that is associated with the instances.

Answer: C

Explanation:

To resolve the issue of EC2 instances not appearing in Systems Manager Fleet Manager, despite the SSM Agent being installed and internet connectivity provided by a NAT gateway, the most critical missing component is usually the correct IAM permissions for the instances.

The **AmazonSSMManagedInstanceCore** AWS managed policy (Option C) grants the necessary permissions

for an EC2 instance to be managed by Systems Manager. This policy allows the Systems Manager Agent (SSM Agent) to perform essential actions such as registering the instance with the Systems Manager service, sending heartbeats, uploading command output, and receiving instructions. Without these permissions, the SSM Agent cannot communicate effectively with the Systems Manager control plane, preventing the instance from appearing in Fleet Manager. Attaching this policy to the EC2 instance profile associated with the instances ensures they have the required authorization while adhering to the principle of least privilege.

Let's evaluate why other options are less suitable:

A. Replace the NAT gateway with a NAT instance: A NAT gateway already provides internet connectivity, which allows instances in private subnets to reach public AWS Systems Manager endpoints. Replacing it with a NAT instance wouldn't fundamentally change the network reachability needed for SSM and doesn't address the core issue of missing IAM permissions. Furthermore, NAT gateways are generally preferred for higher availability and less operational overhead compared to NAT instances.

B. Create a VPC endpoint for Systems Manager and remove NAT gateway routes: While creating a VPC endpoint for Systems Manager is a recommended best practice for enhanced security and to avoid internet traversal, the problem statement indicates that internet connectivity is available through a NAT gateway. If the issue were purely network-related, the NAT gateway should theoretically suffice for reaching public SSM endpoints. More importantly, even with a VPC endpoint, the instances would still require the correct IAM permissions to interact with the Systems Manager service. This option addresses network path, not authentication and authorization.

D. Attach a custom policy that allows all actions to ssm: This policy would grant more than sufficient permissions for the SSM Agent to function. However, it violates the principle of least privilege by granting broad, unnecessary permissions (*ssm:* covers all Systems Manager actions, including managing other services or resources within Systems Manager). The `AmazonSSMManagedInstanceCore` policy (Option C) is specifically designed to provide only the minimum permissions required for an instance to be managed, making it the more secure and appropriate solution.

Therefore, ensuring the EC2 instances have the correct IAM permissions through the `AmazonSSMManagedInstanceCore` policy is the direct and most appropriate solution when the agent is installed and network connectivity is present.

For further research, refer to:

Setting up AWS Systems Manager for EC2 instances: <https://docs.aws.amazon.com/systems-manager/latest/userguide/setup-instance-profile.html>

AWS Managed Policy: `AmazonSSMManagedInstanceCore`: <https://docs.aws.amazon.com/systems-manager/latest/userguide/sysman-policies-iam.html#managed-instance-policy>

Question: 29

CertyIQ

A company has an application that collects notifications from thousands of alarm systems. The notifications include alarm notifications and information notifications. The information notifications include the system arming processes, disarming processes, and sensor status.

All notifications are kept as messages in an Amazon Simple Queue Service (Amazon SQS) queue. Amazon EC2 instances that are in an Auto Scaling group process the messages. A CloudOps engineer needs to implement a solution that prioritizes alarm notifications over information notifications.

Which solution will meet these requirements?

- A. Adjust the Auto Scaling group to scale faster when a high number of messages is in the queue.
- B. Use the Amazon Simple Notification Service (Amazon SNS) fanout feature with Amazon SQS to send the notifications in parallel to all the EC2 instances.
- C. Add an Amazon DynamoDB stream to accelerate the message processing.
- D. Create a queue for alarm notifications and a queue for information notifications. Update the application to

collect messages from the alarm notifications queue first.

Answer: D

Explanation:

The most effective solution to prioritize alarm notifications over information notifications is to implement a queue-based prioritization model, as described in option D.

The core problem is that a single Amazon SQS queue, while providing scalability and decoupling, processes messages in a non-deterministic order for standard queues, or strict FIFO order for FIFO queues. Neither inherently supports content-based prioritization within a single queue without complex application-side logic that might involve re-queuing messages or delaying processing.

Option D addresses this by creating two distinct Amazon SQS queues: one specifically for high-priority alarm notifications and another for lower-priority information notifications. The critical component of this solution is updating the application running on the Amazon EC2 instances to always poll and consume messages from the "alarm notifications" queue first. Only when this high-priority queue is empty will the application then proceed to poll and process messages from the "information notifications" queue.

This approach guarantees that alarm notifications are always processed before any information notifications, effectively implementing the required prioritization. The EC2 instances, even within an Auto Scaling group, will consistently prioritize the retrieval of urgent alarm messages. This strategy leverages the inherent decoupling of SQS queues, allowing different types of messages to reside in separate processing lanes. It is a well-established pattern for message prioritization in distributed systems using SQS.

Let's briefly examine why the other options are less suitable:

A. Adjust the Auto Scaling group to scale faster when a high number of messages is in the queue: While scaling faster improves overall throughput, it doesn't guarantee prioritization. If the single queue contains a mix of alarm and information messages, alarms could still be stuck behind a large backlog of low-priority messages.

B. Use the Amazon Simple Notification Service (Amazon SNS) fanout feature with Amazon SQS to send the notifications in parallel to all the EC2 instances: SNS fanout is used for publishing a single message to multiple subscribers. This doesn't solve message prioritization based on content type within the processing flow; rather, it's for broadcasting messages to potentially different endpoints, not for ordering message consumption from a single pool of messages.

C. Add an Amazon DynamoDB stream to accelerate the message processing: DynamoDB Streams provide a time-ordered sequence of item-level modifications in a DynamoDB table, primarily for event-driven architectures or data replication. They are unrelated to accelerating message processing within an SQS queue or implementing message prioritization in this context.

Therefore, creating separate queues and programming the consumers to prioritize fetching from the high-priority queue is the most direct and robust solution for the stated requirements.

Authoritative Links for further research:

1. **Prioritizing Messages in SQS (AWS Blog Post):** This blog post directly discusses patterns for message prioritization with SQS.

<https://aws.amazon.com/blogs/compute/building-scalable-applications-and-microservices-with-amazon-sqs-and-aws-lambda-part-2-message-prioritization/>

1. **Amazon SQS Developer Guide - How Amazon SQS works:** Provides foundational knowledge on SQS.

<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>

1. **Well-Architected Framework - Operational Excellence Pillar (SQS best practices often fit here):**

While not specific to prioritization, it covers architectural patterns for reliable and efficient operations.

<https://aws.amazon.com/architecture/well-architected/operational-excellence/>

Question: 30

CertyIQ

A company is implementing security and compliance by using AWS Trusted Advisor. The company's CloudOps team is validating the list of Trusted Advisor checks that it can access.

Which factor will affect the quantity of available Trusted Advisor checks?

- A. Whether at least one Amazon EC2 instance is in the running state
- B. The AWS Support plan
- C. An AWS Organizations service control policy (SCP)
- D. Whether the AWS account root user has multi-factor authentication (MFA) enabled

Answer: B

Explanation:

The quantity of available AWS Trusted Advisor checks is directly affected by the AWS Support plan subscribed to by the company. AWS Trusted Advisor provides a set of checks and recommendations across five categories: Cost Optimization, Performance, Security, Fault Tolerance, and Service Limits, helping users provision resources following AWS best practices.

Different AWS Support plans offer varying levels of access to these checks.

1. **Basic and Developer Support plans:** Customers on these plans only have access to a limited set of core security checks and service limit checks. This includes foundational security checks like S3 Bucket Permissions and MFA on Root Account, along with checks for exceeding service limits.
2. **Business and Enterprise Support plans:** These higher-tier plans unlock the full suite of AWS Trusted Advisor checks across all five categories. This provides comprehensive insights into cost savings, performance improvements, enhanced security configurations, improved fault tolerance, and advanced service limit monitoring. For instance, checks related to idle Amazon EC2 instances, underutilized Amazon EBS volumes, high-utilization Amazon EC2 instances, or specific security group rules are only available with these plans.

Therefore, to access the complete list and quantity of Trusted Advisor checks, a company must have at least a Business Support plan. The support plan determines the scope and depth of the recommendations and insights provided by Trusted Advisor, making it a critical factor for organizations aiming for comprehensive compliance and optimization.

Option A is incorrect because while running EC2 instances might trigger some specific checks related to those instances (if available under the support plan), the sheer presence of an instance does not dictate the types or quantity of checks a user can access.

Option C is incorrect because an AWS Organizations Service Control Policy (SCP) primarily controls permissions and can restrict actions across accounts. While an SCP could prevent access to Trusted Advisor entirely, it does not determine the quantity of checks available within the Trusted Advisor service itself, which is a feature tied to the support plan.

Option D is incorrect because enabling MFA on the AWS account root user is a security best practice but has no bearing on the features or checks provided by AWS services like Trusted Advisor.

Authoritative Links:

AWS Trusted Advisor: <https://aws.amazon.com/trusted-advisor/>

AWS Support Plans: <https://aws.amazon.com/premiumsupport/plans/>

AWS Trusted Advisor FAQs (specifically mentioning support plans): <https://aws.amazon.com/trusted-advisor/faqs/> (Refer to "Which AWS Support plans include Trusted Advisor?")

Question: 31

CertyIQ

A CloudOps engineer has successfully deployed a VPC with an AWS CloudFormation template. The CloudOps engineer wants to deploy the same template across multiple accounts that are managed through AWS Organizations.

Which solution will meet this requirement with the LEAST operational overhead?

- A. Assume the OrganizationAccountAccessRole IAM role from the management account. Deploy the template in each of the accounts.
- B. Create an AWS Lambda function to assume a role in each account. Deploy the template by using the AWS CloudFormation CreateStack API call.
- C. Create an AWS Lambda function to query for a list of accounts. Deploy the template by using the AWS CloudFormation CreateStack API call.
- D. Use AWS CloudFormation StackSets from the management account to deploy the template in each of the accounts.

Answer: D

Explanation:

The most effective solution for deploying an AWS CloudFormation template across multiple accounts managed by AWS Organizations with the least operational overhead is to use AWS CloudFormation StackSets from the management account.

AWS CloudFormation StackSets is purpose-built for deploying and managing AWS CloudFormation stacks across multiple AWS accounts and regions from a single administrator account. This service directly addresses the challenge of deploying a common infrastructure template consistently throughout an organization. When integrated with AWS Organizations, StackSets allows an engineer to specify target organizational units (OUs) or individual member accounts for template deployment. A key advantage is its ability to automatically provision and update resources in target accounts without requiring manual intervention in each account. This automation drastically reduces operational overhead compared to individually accessing and deploying in each account. StackSets leverages `AWSCloudFormationStackSetAdministrationRole` in the management account and `AWSCloudFormationStackSetExecutionRole` in target accounts for secure cross-account deployments, simplifying the security model.

Option A, manually assuming the `OrganizationAccountAccessRole` and deploying in each account, involves significant manual effort, is highly prone to human error, and does not scale for a large number of accounts. This approach would introduce substantial operational overhead and delay for every template update or new account onboarding. Options B and C, which involve creating custom AWS Lambda functions to orchestrate deployments using the `CreateStack` API, also introduce unnecessary complexity. While automated, developing, testing, maintaining, and securing a custom Lambda function for this purpose represents a considerable development and ongoing maintenance overhead. Such a custom solution would need to handle error handling, retry logic, account discovery, and role assumption, functionalities inherently provided by StackSets as a fully managed service. StackSets provides a centralized control plane from the management account to create, update, and delete stack instances across all specified target accounts. Furthermore, StackSets can

automatically deploy to new accounts added to a targeted OU, ensuring compliance and consistency from the moment an account is provisioned. This "set it and forget it" capability for initial deployment and ongoing synchronization makes it the most efficient solution for managing infrastructure at an organizational scale, thereby offering the least operational overhead.

Authoritative Links for Further Research:

What is AWS CloudFormation StackSets?

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/what-is-stacksets.html>

Integrate AWS CloudFormation StackSets with AWS Organizations:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacksets-orgs-integration.html>

Walkthrough: Enable trusted access with AWS Organizations:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacksets-orgs-enable-trusted-access.html>

Question: 32

CertyIQ

A company's application is hosted by an internet provider at app.example.com. The company wants to access the application by using www.company.com, which the company owns and manages with Amazon Route 53. Which Route 53 record should be created to address this?

- A. A record
- B. Alias record
- C. CNAME record
- D. Pointer (PTR) record

Answer: C

Explanation:

To enable www.company.com to access an application hosted at app.example.com by an internet provider, a CNAME (Canonical Name) record is the most appropriate choice within Amazon Route 53.

A CNAME record serves to map one domain name, known as the alias, to another domain name, called the canonical name. In this scenario, www.company.com acts as the alias, and app.example.com is its canonical target. When a user's DNS resolver queries www.company.com, Route 53 responds with the CNAME record, directing the resolver to subsequently query app.example.com. The resolver then performs another lookup for app.example.com to obtain its IP address from the hosting provider's DNS servers.

This method is crucial because app.example.com is hosted by an external internet provider. The underlying IP address of app.example.com could change at any time without the company's direct control or notification. By using a CNAME record, any updates to app.example.com's IP address are automatically reflected through its own DNS resolution, requiring no manual changes within Route 53 for www.company.com.

In contrast, an A record maps a domain name directly to an IPv4 address. If an A record were used for www.company.com pointing to app.example.com's current IP, it would break the connection if app.example.com's IP address ever changed, necessitating manual updates and potential downtime.

An Alias record in Route 53 is an AWS-specific extension that allows mapping a domain name to another AWS resource, such as an S3 bucket, an Elastic Load Balancer (ELB), or a CloudFront distribution, or to another record within the same hosted zone. While powerful for integrating with AWS services, it is not primarily designed for directly aliasing to an arbitrary external domain name like app.example.com that is not inherently

an AWS resource.

Finally, a Pointer (PTR) record is used for reverse DNS lookups, mapping an IP address back to a hostname. This is the inverse of the required forward resolution (hostname to IP) needed in this situation. Therefore, the CNAME record provides the most flexible, resilient, and standard DNS solution for aliasing one domain to an externally hosted domain name.

Authoritative Links for Further Research:

AWS Documentation on Resource Record Types:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/ResourceRecordTypes.html>

AWS Documentation on CNAME Records:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/CNAMEs.html>

AWS Documentation on Alias Records:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/HowToAliasRecord.html>

Question: 33

CertyIQ

A company uses Amazon ElastiCache (Redis OSS) to cache application data. A CloudOps engineer must implement a solution to increase the resilience of the cache. The solution also must minimize the recovery time objective (RTO).

Which solution will meet these requirements?

- A. Replace ElastiCache (Redis OSS) with ElastiCache (Memcached).
- B. Create an Amazon EventBridge rule to initiate a backup every hour. Restore the backup when necessary.
- C. Create a read replica in a second Availability Zone. Enable Multi-AZ for the ElastiCache (Redis OSS) replication group.
- D. Enable automatic backups. Restore the backups when necessary.

Answer: C

Explanation:

The most effective solution to increase resilience and minimize Recovery Time Objective (RTO) for an Amazon ElastiCache (Redis OSS) cluster is to implement a Multi-AZ replication strategy. Option C directly addresses this by recommending the creation of a read replica in a second Availability Zone and enabling Multi-AZ for the ElastiCache (Redis OSS) replication group.

Redis replication groups with Multi-AZ enabled provision a primary node in one Availability Zone and one or more read replicas in different Availability Zones. In the event of an outage affecting the primary node or its Availability Zone, ElastiCache automatically promotes one of the read replicas to become the new primary. This automatic failover mechanism is crucial for high availability and significantly enhances resilience.

The key benefit of this approach is its minimal RTO. Automatic failover typically completes within seconds to a few minutes, during which the application can resume operations with minimal interruption. This contrasts sharply with recovery from backups, which involves provisioning a new cluster and restoring data, a process that can take much longer (minutes to hours).

While backups (Options B and D) are essential for data durability and recovery from logical errors like accidental data deletion, they are not optimized for minimizing RTO during infrastructure failures. Restoring from a backup is a reactive, manual or semi-manual process that introduces considerable downtime compared to an automatic failover.

Option A, replacing Redis OSS with ElastiCache (Memcached), is not suitable because Memcached clusters

are fundamentally different. Memcached is designed for scalability and simplicity, but it does not offer persistence or automatic failover with data preservation in the same way Redis replication groups do. If a Memcached node fails, its cached data is lost, and the application must repopulate the cache from the primary data source, potentially impacting performance and not directly addressing the resilience requirement for persistent cache data.

Therefore, configuring a Multi-AZ Redis OSS replication group ensures that if an Availability Zone or primary node becomes unavailable, a healthy replica in another AZ seamlessly takes over, providing continuous cache service with the lowest possible RTO.

Authoritative Links for Further Research:

Amazon ElastiCache for Redis – High Availability and Replication:

<https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/WhatIs.HA.html>

Multi-AZ with Automatic Failover: <https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/AutoFailover.html>

Understanding Recovery Time Objective (RTO) and Recovery Point Objective (RPO): While not specific to ElastiCache, this general concept is fundamental to disaster recovery planning:

<https://aws.amazon.com/blogs/mt/understanding-recovery-time-objective-rto-and-recovery-point-objective-rpo/>

Question: 34

CertyIQ

A company has multiple AWS accounts. A CloudOps engineer uses a sandbox account to create and verify IAM policies for use in a production account. The CloudOps engineer uses AWS CloudFormation to deploy policies to the sandbox account for testing. When tests pass, the CloudOps engineer deploys the policies to production. The CloudOps engineer has configured AWS CloudTrail in both the sandbox account and the production account. The CloudOps engineer wants to detect any changes to the IAM policies after the policies have been deployed by CloudFormation. The CloudOps engineer must receive notifications for any changes to the policies. Which solution will meet these requirements with the LEAST administrative effort?

- A. Configure CloudTrail to send email notifications to the CloudOps engineer when CloudTrail detects changes to the IAM policies.
- B. Create an Amazon EventBridge rule to invoke an AWS Lambda function to check the CloudFormation stack for drift. Configure the function to use Amazon Simple Notification Service (Amazon SNS) to notify the CloudOps engineer if the function detects any drift.
- C. Use AWS Identity and Access Management Access Analyzer to generate a policy based on CloudTrail activity for the IAM role that is attached to the IAM policies in the production account. Compare the results to the IAM policies that are in the sandbox account. Send a notification to the CloudOps engineer if the policies are different.
- D. Store the IAM policies as a JSON document in an Amazon S3 bucket. Use an AWS Lambda function to periodically compare the IAM policies with the JSON document that is stored in the S3 bucket.

Answer: B

Explanation:

Option B, using Amazon EventBridge, AWS Lambda, CloudFormation drift detection, and Amazon SNS, is the most effective solution with the least administrative effort.

Here's a detailed justification:

1. **Directly Addresses the Core Problem:** The problem states that IAM policies are deployed using AWS CloudFormation. CloudFormation's built-in drift detection feature is specifically designed to identify when the actual configuration of a deployed resource (like an IAM policy) deviates from its expected

state as defined in the CloudFormation template. This directly tackles the requirement to detect changes to IAM policies after deployment by CloudFormation.

2. **Leverages Built-in AWS Capabilities:** Instead of custom scripting to compare policies, this solution utilizes CloudFormation's native drift detection functionality. This significantly reduces the development and maintenance overhead compared to building a custom comparison engine.

3. **Low Administrative Effort:**

Amazon EventBridge: An EventBridge rule can be easily configured to trigger on a schedule (e.g., daily or hourly) to initiate the drift check. This is a serverless, managed service requiring minimal setup.

AWS Lambda: A Lambda function can be written with relatively few lines of code to call the AWS SDK's `detect_stack_drift` or `detect_stack_resource_drift` API for the relevant CloudFormation stack(s). Lambda is also serverless, eliminating server provisioning and management.

Amazon SNS: If drift is detected by the Lambda function, it can publish a message to an SNS topic, which can then deliver notifications (e.g., email) to the CloudOps engineer. SNS is a highly available, managed messaging service that is simple to configure for notifications.

4. **Comprehensive and Accurate Detection:** Drift detection precisely identifies what has changed relative to the source of truth (the CloudFormation template). This ensures that only unauthorized or unexpected changes trigger notifications, reducing noise.

5. **Comparison with Other Options:**

A. CloudTrail Notifications: While CloudTrail logs all API calls, configuring it to send specific email notifications for unauthorized IAM policy changes that distinguish from CloudFormation's own updates would require complex EventBridge filtering. This could lead to either too much noise or difficult-to-maintain rules, making it higher administrative effort than leveraging dedicated drift detection.

C. IAM Access Analyzer: Access Analyzer is primarily used for identifying unused permissions, potential security vulnerabilities, or generating policies based on activity. It does not directly compare a deployed policy with an expected template to detect drift, and the proposed comparison logic would be complex and not accurate for this use case.

D. S3 JSON Comparison: This option involves manually managing IAM policy JSON documents in S3 and writing a custom Lambda function to periodically fetch and compare these with the live IAM policies. This essentially re-implements CloudFormation drift detection manually, incurring significant development, testing, and maintenance effort for the custom comparison logic and ensuring the S3 documents remain the definitive source of truth.

In summary, Option B leverages AWS services purpose-built for Infrastructure as Code management and event-driven automation, providing an accurate, automated, and low-overhead solution for detecting and notifying about IAM policy drift.

Authoritative Links for Further Research:

Detecting Drift on CloudFormation Stacks:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-stack-drift.html>

What is Amazon EventBridge? <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>

What is AWS Lambda? <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>

What is Amazon SNS? <https://docs.aws.amazon.com/sns/latest/dg/welcome.html>

analyze the data and create dashboard reports. The company requires that all data access and connections to QuickSight remain within the company's VPC network boundary. Which solution will meet these requirements?

- A. Create an interface VPC endpoint for QuickSight. Configure the endpoint to connect to QuickSight within the VPC by using AWS PrivateLink. Create a manifest file that points to the S3 data. Grant QuickSight permission to access the S3 bucket.
- B. Set up a VPC endpoint for QuickSight. Use an Amazon EC2 instance as a proxy to establish a direct connection between the VPC and QuickSight. Create a manifest file that points to the S3 data. Store the manifest on the EC2 instance. Grant QuickSight permission to access the EC2 instance.
- C. Configure an Amazon S3 VPC gateway endpoint. Route all data from QuickSight through the endpoint to transfer data. Grant QuickSight permission to access the S3 bucket.
- D. Configure a NAT gateway in the company's VPC. Route all data from QuickSight through the NAT gateway to transfer data. Grant QuickSight permission to access the S3 bucket.

Answer: A

Explanation:

The requirement specifies that "all data access and connections to QuickSight remain within the company's VPC network boundary," meaning no traffic should traverse the public internet.

Option A: Create an interface VPC endpoint for QuickSight. Configure the endpoint to connect to QuickSight within the VPC by using AWS PrivateLink. Create a manifest file that points to the S3 data. Grant QuickSight permission to access the S3 bucket.

Interface VPC Endpoints with AWS PrivateLink: This is the core solution that directly addresses the requirement. Interface VPC endpoints, powered by AWS PrivateLink, allow you to establish private connectivity from your Virtual Private Cloud (VPC) to AWS services like QuickSight. When an interface endpoint is configured for QuickSight, all network traffic between your VPC and QuickSight traverses the Amazon internal network securely and does not leave the company's VPC boundary, nor does it pass over the public internet. This ensures that the connection to QuickSight is entirely private.

Manifest File for S3 Data: Amazon QuickSight commonly uses manifest files to specify the location and format of data stored in Amazon S3. This is a standard and necessary step for QuickSight to ingest data from S3.

Granting QuickSight S3 Permissions: For QuickSight to access the confidential data in the S3 bucket, appropriate IAM permissions must be granted. This ensures that QuickSight has the necessary authorization to read the specified S3 objects.

Comprehensive Solution: By combining the private connectivity of an interface endpoint with standard QuickSight data ingestion methods (manifest file) and robust access control (IAM permissions), this solution fully meets all stated requirements for private, secure data analysis.

Why other options are incorrect:

Option B (EC2 Proxy): While an EC2 instance can act as a proxy, using it to establish a "direct connection" between the VPC and QuickSight when an interface endpoint exists for QuickSight is an unnecessary and complex workaround. QuickSight connects directly to its data sources based on configuration, and PrivateLink offers a dedicated, managed solution for private connectivity without introducing an additional layer of compute management.

Option C (S3 VPC Gateway Endpoint only): An S3 VPC gateway endpoint allows resources within your VPC to access S3 privately, without traversing the internet. However, it does not enable an AWS service like QuickSight (which logically operates outside your direct VPC) to connect into your VPC privately. The primary problem is QuickSight's connection into the VPC, not just private access to S3 from within the VPC.

Option D (NAT Gateway): A NAT gateway allows instances in a private subnet to initiate outbound connections to the internet or other AWS services (via their public endpoints). While it prevents inbound

internet connections to private instances, it does not provide private connectivity for external AWS services like QuickSight into the VPC, nor does it ensure that traffic between QuickSight and your VPC remains entirely within the AWS backbone without using public endpoints. Connections would still technically go through public AWS network segments before reaching the NAT gateway's public IP.

Therefore, creating an interface VPC endpoint for QuickSight using AWS PrivateLink is the most direct and architecturally sound approach to ensure all connections and data access remain strictly within the company's VPC network boundary.

Authoritative Links for further research:

1. Connecting to QuickSight with a VPC endpoint:

<https://docs.aws.amazon.com/quicksight/latest/user/connecting-to-quicksight-vpc-endpoint.html>

2. AWS PrivateLink documentation:

<https://aws.amazon.com/privatelink/>

3. VPC Endpoints (General Overview):

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

4. Creating a data set from Amazon S3 files in QuickSight:

<https://docs.aws.amazon.com/quicksight/latest/user/create-s3-data-set.html>

Question: 36

CertyIQ

A company is migrating its production file server to AWS. All data that is stored on the file server must remain accessible if an Availability Zone becomes unavailable or when system maintenance is performed. Users must be able to interact with the file server through the SMB protocol. Users also must have the ability to manage file permissions by using Windows ACLs.

Which solution will meet these requirements?

- A. Create a single AWS Storage Gateway file gateway.
- B. Create an Amazon FSx for Windows File Server Multi-AZ file system.
- C. Deploy two AWS Storage Gateway file gateways across two Availability Zones. Configure an Application Load Balancer in front of the file gateways.
- D. Deploy two Amazon FSx for Windows File Server Single-AZ 2 file systems. Configure Microsoft Distributed File System Replication (DFSR).

Answer: B

Explanation:

The optimal solution for migrating a production file server to AWS with the specified requirements is to create an Amazon FSx for Windows File Server Multi-AZ file system. This service is specifically designed to provide fully managed, highly available, and scalable file storage that is natively compatible with Windows applications and services.

Firstly, Amazon FSx for Windows File Server fully supports the Server Message Block (SMB) protocol, enabling users to interact with the file server using the standard file sharing method expected in a Windows environment. This addresses the requirement for SMB protocol access seamlessly.

Secondly, it provides native support for Windows Access Control Lists (ACLs), allowing users to manage file and folder permissions using familiar tools and ensuring that the existing permission structure from the on-

premises file server can be maintained and managed in AWS. This is crucial for maintaining proper data governance and user access control.

Most critically, the Multi-AZ deployment option directly addresses the high availability requirement. An FSx for Windows File Server Multi-AZ file system automatically provisions and maintains a standby file server in a separate Availability Zone (AZ) from the primary server. Data is synchronously replicated between the primary and standby file servers, ensuring no data loss in the event of a failure. Should the primary file server become unavailable due to an AZ outage or scheduled maintenance, FSx automatically fails over to the standby server, providing continuous data accessibility without manual intervention or downtime for users. This guarantees that the file server remains accessible even when an AZ becomes unavailable or during system maintenance.

In contrast, a single AWS Storage Gateway file gateway (Option A) would represent a single point of failure, failing the high availability requirement. Deploying two Storage Gateway file gateways with an Application Load Balancer (Option C) is unsuitable because an ALB does not support SMB traffic; even with an NLB, managing SMB sessions across independent gateways serving the same underlying storage without additional layers like DFS Namespaces would be complex and prone to consistency issues. Deploying two Amazon FSx Single-AZ 2 file systems with Microsoft Distributed File System Replication (DFSR) (Option D) introduces replication latency and potential for data conflicts, and does not offer the same seamless, automatic, and synchronously replicated failover experience as a native FSx Multi-AZ system, which is purpose-built for this level of high availability within a region.

Therefore, an Amazon FSx for Windows File Server Multi-AZ file system is the most direct, robust, and managed solution to meet all the stated requirements for high availability, SMB access, and Windows ACL support.

Authoritative Links:

Amazon FSx for Windows File Server: <https://aws.amazon.com/fsx/windows/>

FSx for Windows File Server high availability and durability:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/high-availability-durability.html>

AWS Storage Gateway: <https://aws.amazon.com/storagegateway/>

Question: 37

CertyIQ

A company has a web application that is experiencing performance problems many times each night. A root cause analysis reveals sudden increases in CPU utilization that last 5 minutes on an Amazon EC2 Linux instance. A CloudOps engineer must find the process ID (PID) of the service or process that is consuming more CPU. What should the CloudOps engineer do to collect the process utilization information with the LEAST amount of effort?

- A. Configure the Amazon CloudWatch agent procstat plugin to capture CPU process metrics.
- B. Configure an AWS Lambda function to run every minute to capture the PID and send a notification.
- C. Log in to the EC2 instance by using a .pem key each night. Then run the top command.
- D. Use the default Amazon CloudWatch CPU utilization metric to capture the PID in CloudWatch.

Answer: A

Explanation:

The correct answer is A. Configuring the Amazon CloudWatch agent procstat plugin to capture CPU process metrics is the most efficient and least effort solution for a CloudOps engineer to identify the process ID (PID) consuming high CPU on an Amazon EC2 Linux instance.

Here's a detailed justification:

1. **Purpose of the procstat plugin:** The CloudWatch agent, when installed on an EC2 instance, can be configured with various plugins. The procstat plugin is specifically designed to collect detailed system metrics for individual processes. This includes critical information such as CPU utilization percentage, memory usage, and importantly, the Process ID (PID) and process name.
2. **Meeting the requirement:** The problem explicitly asks for the process ID (PID) of the service or process consuming more CPU. The procstat plugin directly addresses this by reporting metrics per process, allowing the engineer to pinpoint the specific culprit.

3. **"Least amount of effort":**

One-time configuration: The CloudWatch agent needs to be installed (if not already present) and its configuration file (config.json or similar) updated to include the procstat plugin. This is a one-time setup effort.

Automated collection: Once configured and the agent is running, it automatically and continuously collects the specified process metrics at defined intervals and sends them to CloudWatch. No manual intervention is required each night.

Centralized monitoring: The data is available in CloudWatch, allowing for easy visualization, dashboard creation, alarming, and historical analysis without needing to log into the instance.

4. **Why other options are less suitable:**

B. Configure an AWS Lambda function to run every minute: This involves significant development effort (writing code to connect to the instance, execute commands like `top` or `ps`, parse their output, extract the PID, and then send a notification). It also introduces complexity in managing permissions and potential connection failures. This is far from the "least amount of effort."

C. Log in to the EC2 instance by using a .pem key each night. Then run the top command: This is a manual, reactive, and highly inefficient approach. It requires the engineer to be awake and available at the exact time the spikes occur, which might be unpredictable. It doesn't scale and is prone to human error and missed events, making it the opposite of "least amount of effort" for ongoing monitoring.

D. Use the default Amazon CloudWatch CPU utilization metric to capture the PID in CloudWatch: The default EC2 CPU utilization metric provides the total CPU usage for the entire instance. It does not provide any information about individual processes or their PIDs. While it can show that the CPU is high, it cannot identify which process is causing it, failing to meet the core requirement of finding the PID.

In summary, the CloudWatch agent with the procstat plugin offers a robust, automated, and low-maintenance solution for granular process-level monitoring, directly addressing the need to find the offending PID with the least operational effort for the CloudOps engineer.

Authoritative Links for Further Research:

Monitoring processes on Linux with the CloudWatch agent:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Install-CloudWatch-Agent-on-EC2-Instance.html#Agent-process-metrics>

CloudWatch Agent Configuration File (procstat section):

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Agent-Configuration-File-Details.html> (Search for procstat within the page)

A company needs to monitor its website's availability to end users. The company needs a solution to provide an Amazon Simple Notification Service (Amazon SNS) notification if the website's uptime decreases to less than 99%. The monitoring must provide an accurate view of the user experience on the website. Which solution will meet these requirements?

- A. Create an Amazon CloudWatch alarm that is based on the website's logs that are published to a CloudWatch Logs log group. Configure the alarm to publish an SNS notification if the number of HTTP 4xx errors and 5xx errors exceeds a specified threshold.
- B. Create an Amazon CloudWatch alarm that is based on the website's published metrics in CloudWatch. Configure the alarm to publish an SNS notification that is based on anomaly detection.
- C. Create an Amazon CloudWatch Synthetics heartbeat monitoring canary. Associate the canary with the website's URL for end users. Create a CloudWatch alarm for the canary. Configure the alarm to publish an SNS notification if the value of the SuccessPercent metric is less than 99%.
- D. Create an Amazon CloudWatch Synthetics broken link checker monitoring canary. Associate the canary with the website's URL for end users. Create a CloudWatch alarm for the canary. Configure the alarm to publish an SNS notification if the value of the SuccessPercent metric is less than 99%.

Answer: C

Explanation:

The correct solution is C because it directly addresses the need for monitoring the website's availability from an end-user perspective and accurately reflecting the user experience.

1. **CloudWatch Synthetics for End-User Experience:** Amazon CloudWatch Synthetics allows users to create canaries, which are configurable scripts that monitor endpoints and APIs using real browsers. This approach simulates actual user traffic and interactions, providing an "accurate view of the user experience" as required by the problem statement. This is crucial because internal server metrics (like CPU utilization or memory usage) or server-side logs (like 4xx/5xx errors from an origin server) do not fully capture what an end-user experiences, such as network latency, DNS resolution issues, or problems with CDN delivery.
2. **Heartbeat Monitoring Canary:** A heartbeat monitoring canary is specifically designed to regularly check a single URL's availability and performance. This makes it ideal for the core requirement of monitoring "website's availability to end users." It performs a simple, frequent check, ensuring the website loads successfully.
3. **SuccessPercent Metric:** CloudWatch Synthetics canaries publish various metrics to CloudWatch, including SuccessPercent. This metric represents the percentage of canary runs that completed successfully. If the website is unavailable or returns an error that prevents the canary from succeeding, the SuccessPercent will decrease.
4. **Meeting the 99% Uptime Threshold:** An Amazon CloudWatch alarm can be configured on the SuccessPercent metric. Setting the alarm to trigger when the SuccessPercent falls below 99% directly fulfills the requirement for an "Amazon SNS notification if the website's uptime decreases to less than 99%."
5. **SNS Notification:** CloudWatch alarms can be configured to publish notifications to an Amazon SNS topic, ensuring the company receives timely alerts when the uptime threshold is breached.
6. **Why other options are less suitable:**

A (Logs-based alarm): While useful for detecting server-side errors, it doesn't provide an end-user view. If the website is completely down and not serving any logs, this alarm might not trigger. It also doesn't account for frontend issues, slow loading times, or CDN problems that affect user experience but don't necessarily generate 4xx/5xx server errors.

B (Anomaly detection on published metrics): This is too vague. If "website's published metrics" refer to internal server metrics, it suffers from the same limitations as option A regarding end-user perspective. Anomaly detection is great for detecting unusual patterns but a direct threshold on SuccessPercent is more precise for an explicit 99% uptime SLA.

D (Broken link checker canary): While using CloudWatch Synthetics, a "broken link checker" canary is designed for a more specific purpose – crawling multiple pages to find broken links and validate content. While it would generate a SuccessPercent, a "heartbeat monitoring canary" is a more direct, efficient, and cost-effective solution specifically tailored for simple website availability/uptime monitoring of a single URL, as implied by the question. The question focuses on overall website availability and uptime, for which a heartbeat check is the most suitable.

Therefore, using a CloudWatch Synthetics heartbeat monitoring canary with an alarm on its SuccessPercent metric directly and accurately meets all the specified requirements, providing an end-user perspective on availability and triggering an SNS notification for uptime breaches.

Authoritative Links for Further Research:

Amazon CloudWatch Synthetics: <https://aws.amazon.com/cloudwatch/features/synthetics/>

Monitoring with Canaries:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Synthetics_Canaries_Monitoring.html

Using Amazon CloudWatch Alarms:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Using_CloudWatch_Alarms.html

Question: 39

CertyIQ

A company uses Amazon EC2 Auto Scaling across multiple Availability Zones. The company must ensure that EC2 instances are provisioned in private subnets.

The company recently optimized its cloud infrastructure by reducing the number of NAT gateways in the company's VPC to one. Some EC2 instances lost internet connectivity after the infrastructure update. A CloudOps engineer must resolve the connectivity issue.

Which solution will meet this requirement?

- A. Replace the existing NAT gateway with a NAT instance in the same subnet.
- B. Update VPC route tables to target the existing NAT gateway for internet traffic.
- C. Update VPC route tables to target an internet gateway for internet traffic.
- D. Add secondary IP addresses to the existing NAT gateway.

Answer: B

Explanation:

The core issue is that EC2 instances residing in private subnets have lost internet connectivity after the company reduced its NAT Gateways to a single instance. Instances in private subnets are designed not to have direct access to the internet. Instead, they rely on a NAT Gateway (or a NAT instance, though less common now) located in a public subnet to facilitate outbound internet connectivity while preventing unsolicited inbound traffic.

The most probable cause for the loss of connectivity is that the route tables associated with the affected private subnets are no longer correctly configured to direct internet-bound traffic (0.0.0.0/0) to the remaining NAT Gateway. When multiple NAT Gateways existed, private subnets might have been configured to use a specific NAT Gateway in their respective Availability Zones. After the reduction, some private subnets' route tables likely still point to a now-removed NAT Gateway or have no valid default route for internet traffic.

Therefore, the solution is to update the VPC route tables. Each private subnet that requires internet access must have a route entry for 0.0.0.0/0 (representing all internet traffic) that targets the ID of the existing NAT Gateway. A single NAT Gateway, when properly provisioned in a public subnet, can serve private subnets across all Availability Zones within the same VPC. Traffic from private subnets in different AZs will route through the VPC network to reach the NAT Gateway.

Let's evaluate the other options:

A. Replace the existing NAT gateway with a NAT instance in the same subnet. This is generally a downgrade. NAT Gateways are AWS-managed, highly available, and provide better performance and fewer operational overheads than self-managed NAT instances. The problem is a routing misconfiguration, not the type of NAT service.

C. Update VPC route tables to target an internet gateway for internet traffic. This is incorrect for private subnets. Instances in private subnets should never directly route to an Internet Gateway for 0.0.0.0/0. That configuration is exclusively for public subnets, which would expose the instances to direct internet access, violating the security principles of a private subnet.

D. Add secondary IP addresses to the existing NAT gateway. Adding secondary IP addresses to a NAT Gateway does not resolve a fundamental routing problem where private subnets cannot reach the NAT Gateway in the first place. It's used for specific advanced networking patterns, not for establishing basic connectivity.

By updating the route tables (Option B), the CloudOps engineer directly addresses the routing configuration gap, restoring internet connectivity for the private EC2 instances through the consolidated NAT Gateway without compromising the private subnet architecture.

Authoritative Links:

What is Amazon VPC? (NAT Gateway section): <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html#nat-gateways>

Route tables for your VPC: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

NAT gateways vs. NAT instances: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-comparison.html>

Question: 40

CertyIQ

A company uses AWS CloudFormation to manage a stack of Amazon EC2 instances on AWS. A CloudOps engineer needs to keep the instances and all of the instances' data, even if someone deletes the stack. Which solution will meet these requirements?

- A. Set the DeletionPolicy attribute to Snapshot for the EC2 instance resource in the CloudFormation template.
- B. Automate backups by using Amazon Data Lifecycle Manager (Amazon DLM).
- C. Create a backup plan in AWS Backup.
- D. Set the DeletionPolicy attribute to Retain for the EC2 instance resource in the CloudFormation template.

Answer: D

Explanation:

The correct solution is D because it directly addresses the requirement to keep the Amazon EC2 instances and their associated data even if the AWS CloudFormation stack is deleted.

1. **CloudFormation DeletionPolicy:** AWS CloudFormation provides a DeletionPolicy attribute that you can apply to resources in your template. This attribute dictates how CloudFormation handles a

resource when the stack that contains it is deleted.

2. **Retain Value:** When you set the DeletionPolicy attribute to Retain for a specific resource, CloudFormation will not delete that resource (or its primary data, like an EC2 instance's root EBS volume) when the stack is deleted. Instead, the resource will be detached from the stack and remain in your AWS account, becoming unmanaged by CloudFormation.
3. **Application to EC2:** For an AWS::EC2::Instance resource, setting DeletionPolicy: Retain ensures that when the CloudFormation stack is deleted, the EC2 instance itself will continue to run, and its attached root Amazon EBS volume (which contains the operating system and instance data) will also be retained, fulfilling the requirement to keep "the instances and all of the instances' data."

Let's evaluate the other options:

A. Set the DeletionPolicy attribute to Snapshot for the EC2 instance resource in the CloudFormation template. This option is incorrect because Snapshot is not a valid DeletionPolicy for an AWS::EC2::Instance resource. The Snapshot policy is primarily for resources like AWS::EC2::Volume or AWS::RDS::DBInstance, where it creates a snapshot of the volume or database before the resource itself is deleted. Even if it were valid, it would only preserve data in a snapshot, not keep the instance running.

B. Automate backups by using Amazon Data Lifecycle Manager (Amazon DLM). Amazon DLM is an excellent service for automating the creation, retention, and deletion of snapshots and AMIs of EBS volumes. While it ensures data backup, it does not prevent the EC2 instance resource from being deleted by CloudFormation when the stack is deleted. If the instance is deleted, you would need to launch a new instance and restore data from a snapshot, which does not "keep the instances" as per the requirement.

C. Create a backup plan in AWS Backup. AWS Backup is a centralized backup service for various AWS resources. Similar to Amazon DLM, it focuses on creating and managing backups for recovery purposes. While highly recommended for data protection, a backup plan in AWS Backup would not prevent CloudFormation from deleting the EC2 instance and its volumes upon stack deletion. You would still need to restore from a backup to a new instance if the original was deleted.

Therefore, DeletionPolicy: Retain is the only solution that directly ensures the EC2 instances and their data remain active and available in the AWS account after the CloudFormation stack is deleted.

Authoritative Links for further research:

AWS CloudFormation Resource DeletionPolicy Attribute:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-attribute-deletionpolicy.html>

AWS::EC2::Instance CloudFormation documentation:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-properties-ec2-instance.html>

Question: 41

CertyIQ

A CloudOps engineer is preparing to deploy an application to Amazon EC2 instances that are in an Auto Scaling group. The application requires dependencies to be installed. Application updates are issued weekly. The CloudOps engineer needs to implement a solution to incorporate the application updates on a regular basis. The solution also must conduct a vulnerability scan during Amazon Machine Image (AMI) creation. What is the MOST operationally efficient solution that meets these requirements?

- A. Create a script that uses Packer. Schedule a cron job to run the script.
- B. Install the application and its dependencies on an EC2 instance. Create an AMI of the EC2 instance.
- C. Use EC2 Image Builder with a custom recipe to install the application and its dependencies.
- D. Invoke the EC2 CreateImage API operation by using an Amazon EventBridge scheduled rule.

Answer: C

Explanation:

The most operationally efficient solution is **C. Use EC2 Image Builder with a custom recipe to install the application and its dependencies.**

Here's a detailed justification:

EC2 Image Builder is a fully managed AWS service specifically designed to automate the creation, maintenance, validation, and testing of Amazon Machine Images (AMIs). This service directly addresses all the stated requirements in the most operationally efficient manner.

First, to handle the installation of the application and its required dependencies, EC2 Image Builder allows the creation of **custom recipes**. These recipes define the build process, including the base operating system image, and a sequence of components. These components can be pre-built by AWS or custom scripts that install specific software, configure settings, and, crucially, install the application and its dependencies onto the base image. This directly addresses the need for dependency installation.

Second, for incorporating weekly application updates, EC2 Image Builder pipelines can be **scheduled** to run on a regular basis. A CloudOps engineer can configure the pipeline to automatically start a new image build weekly, ensuring that the latest application updates are consistently integrated into a fresh AMI without manual intervention. This eliminates the need for external scheduling mechanisms like cron jobs on separate servers, significantly boosting operational efficiency.

Third, a critical requirement is to **conduct a vulnerability scan during AMI creation**. EC2 Image Builder provides robust **validation and testing phases** within its pipelines. It can be integrated with AWS services like Amazon Inspector, which automatically performs vulnerability assessments on the newly built image. This integration allows the CloudOps engineer to define criteria for passing a build based on scan results, preventing deployment of images with known vulnerabilities and satisfying the security requirement as part of the automated build process.

Compared to other options:

Option A (Packer with cron job) requires managing a separate build server to run Packer and manually integrating vulnerability scanning, increasing operational overhead. While effective, it's not as fully managed as EC2 Image Builder.

Option B (Manual installation and AMI creation) is highly inefficient, prone to errors, and not scalable for regular updates or built-in scanning.

Option D (EventBridge with CreateImage API) only creates an image of an existing instance and does not address the installation of dependencies, application updates, or vulnerability scanning during the build process itself.

In summary, EC2 Image Builder offers a complete, AWS-native, and highly automated solution that centralizes AMI lifecycle management, including dependency installation via custom recipes, weekly update scheduling, and integrated vulnerability scanning, making it the most operationally efficient choice.

Authoritative Links for further research:

AWS EC2 Image Builder: <https://aws.amazon.com/image-builder/>

How EC2 Image Builder works: <https://docs.aws.amazon.com/imagebuilder/latest/userguide/how-image-builder-works.html>

Creating an EC2 Image Builder image recipe:

<https://docs.aws.amazon.com/imagebuilder/latest/userguide/image-recipes.html>

Validation and test components in Image Builder:

<https://docs.aws.amazon.com/imagebuilder/latest/userguide/test-components.html>

Question: 42

A company is planning to host an application on a set of Amazon EC2 instances that are distributed across multiple Availability Zones. The application must be able to scale to millions of requests each second.

A CloudOps engineer must design a solution to distribute the traffic to the EC2 instances. The solution must be optimized to handle sudden and volatile traffic patterns while using a single static IP address for each Availability Zone.

Which solution will meet these requirements?

- A. Amazon Simple Queue Service (Amazon SQS) queue
- B. Application Load Balancer
- C. AWS Global Accelerator
- D. Network Load Balancer

Answer: D**Explanation:**

The Network Load Balancer (NLB) is the optimal solution for these requirements. NLB operates at Layer 4 (TCP/UDP), making it exceptionally well-suited for handling extremely high throughput, ultra-low latency, and sudden, volatile traffic patterns that can scale to millions of requests per second. Its architecture is designed for performance, forwarding client traffic directly to target EC2 instances without additional processing overhead, which is crucial for such demanding applications.

A key differentiator and explicit requirement met by NLB is its ability to provide a single static IP address for each Availability Zone where it is deployed. These static, private IP addresses or Elastic IP addresses ensure a consistent and stable endpoint for client applications, simplifying network configurations and allowing clients to directly target specific AZs if needed. This direct IP access, combined with its flow-based load balancing, ensures efficient distribution of traffic even during massive and unpredictable spikes.

In contrast, an Amazon Simple Queue Service (SQS) queue is a messaging service used for decoupling application components and asynchronous processing, not for direct, real-time traffic distribution to EC2 instances. An Application Load Balancer (ALB), while highly capable at Layer 7 (HTTP/HTTPS), introduces more processing overhead for advanced routing and SSL/TLS termination, which might not be ideal for the absolute highest performance at millions of requests per second and could have a different IP scaling model compared to NLB's static AZ IP feature. AWS Global Accelerator is designed for optimizing global traffic routing and providing static anycast IPs at edge locations, primarily sitting in front of regional load balancers or EC2 instances with Elastic IPs, rather than being the direct regional traffic distributor to EC2 instances as specified.

Therefore, the Network Load Balancer's unparalleled performance, capacity to handle volatile traffic patterns without pre-warming, and its unique provision of static IP addresses per Availability Zone make it the perfect fit for the company's demanding application.

Authoritative Links for further research:

AWS Network Load Balancer: [https://aws.amazon.com/elasticloadbalancing/network-load-balancer/ElasticLoadBalancingFeatures\(Overview\):](https://aws.amazon.com/elasticloadbalancing/network-load-balancer/ElasticLoadBalancingFeatures(Overview):)

<https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/what-is-elb.html>

AWS Blog on NLB and ALB: <https://aws.amazon.com/blogs/aws/new-elastic-load-balancing-type-network-load-balancer/>

Question: 43

A company has a stateful web application that is hosted on Amazon EC2 instances in an Auto Scaling group. The instances run behind an Application Load Balancer (ALB) that has a single target group. The ALB is configured as the origin in an Amazon CloudFront distribution. Users are reporting random logouts from the web application. Which combination of actions should a CloudOps engineer take to resolve this problem? (Choose two.)

- A. Change to the least outstanding requests algorithm on the ALB target group,
- B. Configure cookie forwarding in the CloudFront distribution cache behavior.
- C. Configure header forwarding in the CloudFront distribution cache behavior.
- D. Enable group-level stickiness on the ALB listener rule.
- E. Enable sticky sessions on the ALB target group

Answer: BE

Explanation:

Stateful web applications require session persistence, ensuring that all requests from a single user are consistently routed to the same backend instance to maintain their unique session data. In this scenario, random logouts indicate a breakdown in this persistence, likely due to requests being distributed across different EC2 instances in the Auto Scaling group without proper session affinity. The Application Load Balancer (ALB) behind Amazon CloudFront distributes incoming requests, but by default, it does not guarantee session stickiness. Furthermore, CloudFront, as a content delivery network, can strip or ignore cookies by default when forwarding requests to the origin, which is crucial for identifying user sessions.

To resolve this, configuring cookie forwarding in the CloudFront distribution cache behavior (B) is essential. By default, CloudFront might not forward the HTTP Cookie header from the viewer to the origin (ALB), preventing the ALB from receiving critical session-identifying information, such as the AWSELB stickiness cookie or the application's own session cookie. Enabling this forwarding ensures that the session context, encapsulated in cookies, reaches the ALB, allowing it to make informed routing decisions.

Concurrently, enabling sticky sessions on the ALB target group (E) is paramount. This feature, also known as session affinity, ensures that once a user's request is routed to a specific EC2 instance, all subsequent requests from that user during the session lifetime are directed to the same instance. The ALB achieves this by injecting a load balancer generated cookie (e.g., AWSELB) into the HTTP response, which the client then includes in future requests. This explicit configuration on the ALB target group directly prevents session loss by maintaining consistent routing to the instance holding the user's state.

The combination of these two actions provides a robust solution: CloudFront ensures session cookies are passed to the ALB, and the ALB then uses those cookies (or its own generated ones) to consistently route requests to the correct backend instance. Changing to the least outstanding requests algorithm (A) only alters load distribution, not session stickiness. Configuring general header forwarding (C) is too broad and less specific than cookie forwarding (B) for session management. Option D is poorly phrased; sticky sessions are configured at the target group level, which is accurately captured by option E. Thus, actions B and E together ensure that session state is preserved end-to-end, eliminating random user logouts.

For more details on ALB sticky sessions, refer to the AWS documentation on Target Group Attributes:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html#target-group-attributes> Information on CloudFront cookie forwarding can be found in the AWS documentation on Cache Policies and Headers:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-cache-policies.html#cache-policy-cache-key-settings>

Question: 44

A global company uses an organization in AWS Organizations to manage multiple AWS accounts. To comply with regulations, the company deploys workload environments to five AWS Regions. The company has a separate AWS account for each Region.

The company needs to connect every environment's VPC to a central shared VPC that serves as a directory and to a shared monitoring VPC. The shared accounts are each in separate AWS accounts.

Which solution will meet these requirements?

- A. Create a transit gateway in the central shared AWS account. Share the transit gateway with the company's AWS accounts. Connect all VPCs to the central transit gateway.
- B. Create a separate transit gateway in every Region where the company has deployed resources. Share the transit gateways with company's AWS accounts. Connect the VPC in each Region to the transit gateway that is in the same Region. Peer the transit gateways. Create appropriate routes in all route tables.
- C. Create a virtual private gateway for the shared VPCs. Create a customer gateway for the workload VPCs. Configure an AWS Site-to-Site VPN connection between the directory VPC, the monitoring VPC, and every workload VPC.
- D. Create VPC peering connections between the central shared VPC, the shared monitoring VPC, and every workload VPC.

Answer: B

Explanation:

The correct solution is **B** because it leverages AWS Transit Gateway (TGW) to create a robust, scalable, and manageable network architecture across multiple AWS Regions and accounts.

Here's a detailed justification:

The company needs to connect workload VPCs across five AWS Regions to central shared VPCs (directory and monitoring), which are also in separate AWS accounts. This scenario demands a solution that supports multi-account, multi-region connectivity with high scalability and simplified management.

1. **Regional Nature of Transit Gateway:** AWS Transit Gateways are regional resources. Therefore, to connect VPCs residing in different AWS Regions, a separate Transit Gateway must be deployed in each of those Regions. Option B correctly identifies this by stating, "Create a separate transit gateway in every Region where the company has deployed resources." This ensures that each regional workload VPC can attach to a TGW within its own Region.
2. **Multi-Account Connectivity with AWS Resource Access Manager (RAM):** Transit Gateways can be shared across multiple AWS accounts within an AWS Organization using AWS Resource Access Manager (RAM). This allows the workload VPCs (each in a separate account for each Region) to attach to the respective regional Transit Gateways. Similarly, the central shared VPC and shared monitoring VPCs can attach to a TGW in their Region, regardless of which account they reside in.
3. **Hub-and-Spoke Topology:** Attaching all relevant VPCs (workload, directory, monitoring) within a Region to a single Transit Gateway creates a hub-and-spoke network topology. This simplifies routing by centralizing network connectivity, as traffic between any attached VPCs (in the same Region) traverses the TGW.
4. **Cross-Region Connectivity with TGW Peering:** To connect Transit Gateways across different AWS Regions, TGW peering connections are utilized. Since the workload environments are deployed across five Regions and need to connect to shared services that might not be co-located in all those Regions, or even to workload VPCs in other Regions, TGW peering is essential. This allows seamless communication between VPCs attached to different TGWs across the globe, providing a global network backbone.

5. **Scalability and Management:** This TGW-based approach significantly scales better than alternatives. Instead of managing numerous point-to-point connections, you manage a few regional TGWs and their peering connections. This reduces operational overhead and simplifies network changes as more VPCs or accounts are added.

Why other options are less suitable or incorrect:

A. Create a transit gateway in the central shared AWS account. Share the transit gateway with the company's AWS accounts. Connect all VPCs to the central transit gateway. This is incorrect because a single Transit Gateway cannot span multiple AWS Regions. You cannot directly connect a VPC in one Region to a TGW in another Region. Cross-region connectivity requires TGW peering.

C. Create a virtual private gateway for the shared VPCs. Create a customer gateway for the workload VPCs. Configure an AWS Site-to-Site VPN connection between the directory VPC, the monitoring VPC, and every workload VPC. While AWS Site-to-Site VPNs can connect VPCs, this approach is designed primarily for connecting on-premises networks to AWS VPCs. Using it for inter-VPC communication within AWS across many accounts and regions creates a complex, unscalable mesh of VPN tunnels, is difficult to manage, and typically has higher latency and cost than native AWS networking solutions like TGW.

D. Create VPC peering connections between the central shared VPC, the shared monitoring VPC, and every workload VPC. VPC peering connections are 1:1, non-transitive, and must be established for every pair of VPCs that need to communicate directly. In a scenario with many workload VPCs across regions and multiple shared VPCs, this would lead to an unmanageable number of peering connections and complex route tables. More importantly, it's not transitive, meaning a workload VPC cannot reach another shared VPC via an intermediate shared VPC that it is peered with. This violates the hub-and-spoke requirement for central services.

In summary, Option B provides a robust, scalable, and manageable network solution using AWS Transit Gateway's core capabilities, including regional deployments, multi-account sharing, and cross-region peering, to meet the complex connectivity requirements of a global enterprise.

Authoritative Links for Further Research:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Transit Gateway Peering: <https://docs.aws.amazon.com/vpc/latest/tgw/transit-gateway-peering.html>

Share your Transit Gateway with AWS Resource Access Manager (RAM):

<https://docs.aws.amazon.com/vpc/latest/tgw/tgw-account-sharing.html>

VPC peering vs. AWS Transit Gateway: <https://aws.amazon.com/blogs/architecture/vpc-peering-vs-aws-transit-gateway/>

Question: 45

CertyIQ

A company stores critical information in an Amazon RDS for PostgreSQL database. The company notices degraded performance, high CPU utilization, increased query latency, and connection timeouts during peak shopping hours. The company also identifies surges in user connections during peak hours. The connection surges affect the read performance of the database.

The company wants to resolve the database performance issues.

Which combination of steps will meet this requirement? (Choose two.)

- A. Use Amazon RDS Performance Insights to analyze the SQL queries that most affect database performance. Update the SQL queries based on the analysis findings.
- B. Use Amazon CloudWatch Logs Insights to analyze the database queries and identify performance bottlenecks. Update the queries based on the analysis findings.
- C. Use Amazon RDS for PostgreSQL with a single Availability Zone.
- D. Disable connection pooling entirely to ensure that all user connections are treated equally, even during peak hours.

Answer: AE

Explanation:

The company's database performance issues, characterized by degraded performance, high CPU utilization, increased query latency, and connection timeouts, are primarily attributed to surges in user connections during peak hours affecting read performance. To effectively resolve these challenges, a multi-faceted approach addressing both internal database efficiency and external connection management is required.

Option A (Use Amazon RDS Performance Insights to analyze the SQL queries that most affect database performance. Update the SQL queries based on the analysis findings.) is a crucial step. Amazon RDS Performance Insights is a powerful monitoring tool designed specifically to help users understand and visualize the database load. It identifies the top SQL queries, wait events, hosts, and users that contribute most to the database's active sessions, providing granular data to pinpoint performance bottlenecks. High CPU utilization and increased query latency often stem from inefficient or poorly optimized SQL queries. By analyzing these queries through Performance Insights and subsequently updating them (e.g., adding appropriate indexes, rewriting complex joins, optimizing data retrieval patterns), the overall load on the database can be significantly reduced, leading to improved performance. This addresses the root cause of internal processing inefficiencies.

Option E (Implement RDS Proxy with connection pooling.) directly addresses the problem of connection surges and associated connection timeouts. AWS RDS Proxy is a fully managed, highly available database proxy that sits between your application and the RDS database. Its primary function is to efficiently pool and reuse database connections, drastically reducing the overhead on the database instance from frequent connection establishments and terminations. During peak shopping hours with high user connection surges, RDS Proxy prevents the database from being overwhelmed by managing a smaller, more stable set of database connections on behalf of the application. This mitigates connection timeouts, reduces CPU utilization related to connection management, and improves the database's ability to handle high concurrency, thereby enhancing read performance.

Combining these two steps offers a comprehensive solution. Optimizing inefficient queries (A) improves the database's internal processing capacity, while implementing RDS Proxy (E) externalizes and manages connection overhead, shielding the database from connection spikes. Together, they tackle both the efficiency of individual operations and the scalability of connection handling, leading to a robust and performant database solution.

Other options are less suitable:

Option B (CloudWatch Logs Insights) can analyze logs but is not as specialized or effective as Performance Insights for granular SQL query analysis and identifying performance bottlenecks related to active sessions.

Option C (Single Availability Zone) is irrelevant to performance issues; it concerns high availability and durability, and typically Multi-AZ is recommended for production workloads.

Option D (Disable connection pooling) would exacerbate the problem by increasing the overhead of establishing new connections, directly worsening connection timeouts and CPU utilization during peak loads.

Authoritative Links for Further Research:

Amazon RDS Performance Insights: <https://aws.amazon.com/rds/performance-insights/>

Amazon RDS Proxy: <https://aws.amazon.com/rds/proxy/>

A company runs a web application on three Amazon EC2 instances behind an Application Load Balancer (ALB). The company notices that random periods of increased traffic cause a degradation in the application's performance. A CloudOps engineer must scale the application to meet the increased traffic. Which solution meets these requirements?

- A. Create an Amazon CloudWatch alarm to monitor application latency and increase the size of each EC2 instance if the desired threshold is reached.
- B. Create an Amazon EventBridge rule to monitor application latency and add an EC2 instance to the ALB if the desired threshold is reached.
- C. Deploy the application to an Auto Scaling group of EC2 instances with a target tracking scaling policy. Attach the ALB to the Auto Scaling group.
- D. Deploy the application to an Auto Scaling group of EC2 instances with a scheduled scaling policy. Attach the ALB to the Auto Scaling group.

Answer: C

Explanation:

Option C is the most effective and AWS-recommended solution for dynamically scaling a web application to meet random periods of increased traffic.

An **Auto Scaling group (ASG)** is a fundamental AWS service designed to automatically launch and terminate Amazon EC2 instances based on defined policies, ensuring application availability and allowing for elastic scaling. This directly addresses the need to scale out when traffic increases and in when it decreases, without manual intervention.

The key component in this solution is the **target tracking scaling policy**. This policy enables dynamic scaling by maintaining a specified metric (such as average CPU utilization, average network I/O, or Application Load Balancer request count per target) at a target value. For instance, if the target is 70% CPU utilization, the ASG will automatically add or remove instances to keep the average CPU utilization close to that target. This makes it perfectly suited for handling random and unpredictable traffic spikes, ensuring performance degradation is prevented by proactively adding capacity.

Attaching the **Application Load Balancer (ALB)** to the Auto Scaling group ensures that new instances launched by the ASG are automatically registered with the ALB, and terminated instances are de-registered. This seamless integration guarantees that incoming traffic is always distributed efficiently across the current pool of healthy, available instances.

In contrast, option A, which suggests increasing the size of each EC2 instance (vertical scaling), is less suitable. Vertical scaling is limited, causes downtime during the instance type change, and doesn't offer the granular, cost-effective elasticity of horizontal scaling for web applications. Option B, using an Amazon EventBridge rule to add an EC2 instance, would require complex custom automation to replicate the robust features of an ASG, such as instance health checks, replacement, and scaling down, making it less efficient and more prone to errors. Option D, employing a scheduled scaling policy, is designed for predictable traffic patterns (e.g., daily peak hours). It would be ineffective for random traffic spikes, as it would either over-provision resources unnecessarily or under-provision, leading to continued performance issues.

Therefore, deploying the application within an Auto Scaling group utilizing a target tracking scaling policy, integrated with an ALB, provides a fully automated, resilient, and cost-effective solution that directly addresses the problem of performance degradation during random periods of increased traffic.

Authoritative Links for Further Research:

What is AWS Auto Scaling?<https://aws.amazon.com/autoscaling/>

Target tracking scaling policies for Amazon EC2 Auto

Scaling:<https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html>

Question: 47

A company's reporting job that used to run in 15 minutes is now taking an hour to run. An application generates the reports. The application runs on Amazon EC2 instances and extracts data from an Amazon RDS for MySQL database.

A CloudOps engineer checks the Amazon CloudWatch dashboard for the RDS instance and notices that the Read IOPS metrics are high, even when the reports are not running. The CloudOps engineer needs to improve the performance and the availability of the RDS instance.

Which solution will meet these requirements?

- A. Configure an Amazon ElastiCache cluster in front of the RDS instance. Update the reporting job to query the ElastiCache cluster.
- B. Deploy an RDS read replica. Update the reporting job to query the reader endpoint.
- C. Create an Amazon CloudFront distribution. Set the RDS instance as the origin. Update the reporting job to query the CloudFront distribution.
- D. Increase the size of the RDS instance.

Answer: B

Explanation:

The correct solution is B because it effectively addresses both the performance and availability requirements by leveraging a fundamental database scaling pattern in AWS.

A company's reporting job running on Amazon EC2 instances extracts data from an Amazon RDS for MySQL database. The significant slowdown from 15 minutes to an hour, coupled with high Read IOPS even when reports are not running, indicates a persistent read bottleneck on the primary RDS instance. This implies that other applications are also generating considerable read load, leading to resource contention.

Option B: Deploy an RDS read replica. Update the reporting job to query the reader endpoint. This solution directly addresses the identified problem.

1. **Performance:** An Amazon RDS read replica provides a separate database instance that asynchronously replicates data from the primary instance. By directing the read-intensive reporting job to query this read replica, the read workload is offloaded from the primary RDS instance. This reduces the load on the primary, allowing it to perform better for other applications, and simultaneously allows the reporting job to execute faster on its dedicated replica instance without competing for resources with transactional workloads. This is a common and effective horizontal scaling strategy for read-heavy databases.
2. **Availability:** Deploying a read replica enhances the availability of the primary RDS instance by isolating read-intensive workloads. If the reporting job were to cause a spike in resource utilization, it would primarily affect the read replica, not the primary instance that critical applications depend on for transactional operations. Furthermore, read replicas can be promoted to standalone instances in disaster recovery scenarios, offering a layer of data availability (though with potential data staleness). This architectural separation improves the overall resilience and reliability of the database environment.

Why other options are less suitable:

A. Configure an Amazon ElastiCache cluster in front of the RDS instance. Update the reporting job to query the ElastiCache cluster: While ElastiCache can improve read performance for frequently accessed data, it

primarily functions as an in-memory cache for specific, often simple, data objects. Reporting jobs typically involve complex analytical queries over potentially large datasets, which are not usually ideal for simple caching without significant application re-architecture and cache invalidation logic. It's a more complex solution that may not fit the nature of reporting queries as effectively as a read replica for a full dataset.

C. Create an Amazon CloudFront distribution. Set the RDS instance as the origin. Update the reporting job to query the CloudFront distribution: CloudFront is a Content Delivery Network (CDN) designed for caching and delivering static and dynamic web content (HTTP/HTTPS) closer to users. It cannot directly act as a proxy or cache for raw SQL queries to an RDS database instance. This option is fundamentally incompatible with how databases are accessed and queried.

D. Increase the size of the RDS instance: Increasing the instance size (vertical scaling) might offer temporary relief by providing more CPU, memory, and potentially higher IOPS limits. However, it's generally more expensive, doesn't provide workload isolation, and may not resolve the fundamental issue of contention if multiple applications are driving high read load. It's a less scalable and less resilient solution compared to horizontally scaling reads with a read replica, especially when the problem is clearly identifiable as a read bottleneck.

Therefore, deploying an RDS read replica is the most appropriate and architecturally sound solution to improve both the performance of the reporting job and the overall availability of the primary RDS instance.

Authoritative Links for further research:

Amazon RDS Read Replicas:

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_ReadReplicas.html

AWS Well-Architected Framework - Performance Efficiency Pillar:

<https://aws.amazon.com/architecture/well-architected/> (Refer to scaling strategies for databases)

AWS Well-Architected Framework - Reliability Pillar: <https://aws.amazon.com/architecture/well-architected/> (Refer to multi-AZ deployments and read replicas for resilience)

Question: 48

CertyIQ

A company that runs multiple workloads on AWS wants to enhance its security posture by implementing DNS-based threat protection. The company must block DNS-based attacks. Which solution will meet this requirement?

- A. Deploy AWS Shield Advanced to filter and block malicious DNS queries. Set up domain filtering policies.
- B. Use AWS WAF to inspect DNS traffic for malicious domains. Create custom rules to block known threats.
- C. Configure Amazon Route 53 Resolver to forward DNS queries to Route 53 Resolver DNS Firewall Advanced to detect and filter threats.
- D. Configure AWS Config to monitor DNS queries and DNS traffic patterns. Use an AWS Lambda function to prevent access to malicious domains.

Answer: C

Explanation:

The most effective solution for a company running multiple AWS workloads to implement DNS-based threat protection and block DNS-based attacks is to configure Amazon Route 53 Resolver DNS Firewall. This service is purpose-built to provide granular control over the DNS queries originating from Amazon Virtual Private Clouds (VPCs). Route 53 Resolver is the default DNS resolver for all VPCs, and DNS Firewall seamlessly integrates with it to inspect outbound DNS queries.

The DNS Firewall operates by allowing administrators to create rules and rule groups that detect and filter threats. These rules can specify domain names to block, allow, or alert on, based on various criteria. It can

leverage AWS-managed domain lists, which are continuously updated with known malicious domains associated with botnets, malware command and control servers, phishing sites, and data exfiltration attempts. Furthermore, customers can create custom domain lists to block or allow specific domains pertinent to their security policies.

By intercepting DNS queries at the Resolver level before they leave the VPC, DNS Firewall effectively prevents instances and applications from resolving malicious domains. This proactive approach significantly enhances the security posture by disrupting the kill chain of many cyberattacks that rely on DNS resolution for communication or payload delivery. It directly addresses the requirement to block DNS-based attacks by stopping communication with known threat infrastructure.

In contrast, other options are not designed for this specific type of threat protection. AWS Shield Advanced primarily provides DDoS protection for applications and services, including Route 53 hosted zones, but it does not filter outbound DNS queries from within VPCs for malicious domains. AWS WAF (Web Application Firewall) inspects HTTP/S traffic to protect web applications at Layer 7 and is not designed to inspect or filter raw DNS traffic (UDP/TCP port 53 queries). AWS Config monitors resource configurations and changes, while an AWS Lambda function can automate tasks; however, using them for real-time, proactive DNS query filtering and blocking is not their primary purpose and would be significantly more complex and less efficient than a dedicated DNS firewall service. Therefore, Route 53 Resolver DNS Firewall is the optimal, native AWS solution for DNS-based threat protection.

Authoritative Links for further research:

Amazon Route 53 Resolver DNS Firewall: <https://aws.amazon.com/route53/resolver/features/dns-firewall/>

AWS Shield Advanced: <https://aws.amazon.com/shield/>

AWS WAF: <https://aws.amazon.com/waf/>

AWS Config: <https://aws.amazon.com/config/>

Question: 49

CertyIQ

A CloudOps engineer wants to share a copy of a production database with a migration account. The production database is hosted on an Amazon RDS DB instance and is encrypted at rest with an AWS Key Management Service (AWS KMS) key that has an alias of production-rds-key.

What must the CloudOps engineer do to meet these requirements with the LEAST administrative overhead?

- A. Take a snapshot of the RDS DB instance in the production account. Amend the KMS key policy of the production-rds-key KMS key to give access to the migration account's root user. Share the snapshot with the migration account.
- B. Create an RDS read replica in the migration account. Configure the KMS key policy to replicate the production-rds-key KMS key to the migration account.
- C. Take a snapshot of the RDS DB instance in the production account. Share the snapshot with the migration account. In the migration account, create a new KMS key that has an identical alias.
- D. Use native database toolsets to export the RDS DB instance to Amazon S3. Create an S3 bucket and an S3 bucket policy for cross-account access between the production account and the migration account. Use native database toolsets to import the database from Amazon S3 to a new RDS DB instance.

Answer: A

Explanation:

Option A presents the most efficient and least administratively burdensome method for sharing an encrypted Amazon RDS DB instance with another AWS account.

First, taking a snapshot of the RDS DB instance in the production account is the standard, low-overhead way

to create a point-in-time copy of the database. This snapshot retains the encryption status and refers to the original AWS Key Management Service (AWS KMS) key, production-rds-key.

Next, for cross-account sharing of an encrypted snapshot, it is a fundamental requirement that the target account (migration account) is granted decryption permissions on the original KMS key. Amending the key policy of production-rds-key to give kms:Decrypt and kms:CreateGrant access to the migration account's root user (or a specific IAM role/user within that account) directly addresses this. This allows the migration account to use the key to decrypt the snapshot when it attempts to restore it.

Finally, sharing the snapshot with the migration account is a native RDS feature. Once the KMS key policy is correctly configured, the migration account can accept the shared snapshot and restore it into a new RDS DB instance. During restoration, the migration account will typically specify its own KMS key for the new instance's encryption, but it still relies on the original key's permission for the initial decryption of the shared snapshot. This process leverages native AWS services and features, minimizing manual steps and potential errors.

Options B and C are incorrect because they misunderstand how KMS keys and cross-account access work. Option B incorrectly suggests "replicating" a KMS key, which is not a KMS feature; you grant access to a key, not replicate it. Option C incorrectly assumes that creating a new key with the same alias in the target account grants decryption permissions; KMS keys are unique resources identified by their ARN, not just their alias.

Option D, while technically possible, involves significantly higher administrative overhead. It requires using native database tools to export the data, uploading it to Amazon S3 (requiring S3 bucket creation and cross-account policies), and then importing it into a new RDS instance. This is a manual, multi-step logical backup and restore process that is far more complex and time-consuming than leveraging AWS's native encrypted snapshot sharing capabilities.

Therefore, Option A directly leverages the integrated capabilities of Amazon RDS and AWS KMS for encrypted resource sharing, making it the solution with the least administrative overhead.

Authoritative Links:

Sharing an encrypted snapshot:

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ShareSnapshot.html#USER_ShareSnapshot.Encrypted

Allowing users in other accounts to use a KMS key:

<https://docs.aws.amazon.com/kms/latest/developerguide/key-policy-users.html>

Cross-account access with KMS: <https://docs.aws.amazon.com/kms/latest/developerguide/control-access.html#grant-cross-account-access>

Question: 50

CertyIQ

A company has a critical serverless application that uses multiple AWS Lambda functions. Each Lambda function generates 1 GB of log data daily in its own Amazon CloudWatch Logs log group. The company's security team asks for a count of application errors, grouped by type, across all of the log groups. What should a CloudOps engineer do to meet this requirement?

- A. Perform a CloudWatch Logs Insights query that uses the stats command and count function.
- B. Perform a CloudWatch Logs search that uses the groupby keyword and count function.
- C. Perform an Amazon Athena query that uses the SELECT and GROUP BY keywords.
- D. Perform an Amazon RDS query that uses the SELECT and GROUP BY keywords.

Answer: A

Explanation:

The most effective approach for a CloudOps engineer to meet the requirement of counting application errors, grouped by type, across multiple AWS Lambda function log groups is to leverage Amazon CloudWatch Logs Insights.

CloudWatch Logs Insights is a powerful, interactive, and serverless log analytics service specifically designed to query, analyze, and visualize log data from CloudWatch Logs. It allows engineers to search and analyze log data efficiently without needing to export it or set up additional infrastructure.

Here's why option A is the correct choice:

1. **Native Integration and Multi-Log Group Querying:** Logs Insights is purpose-built for CloudWatch Logs. Crucially, it supports querying across multiple selected log groups simultaneously, directly addressing the need to analyze errors "across all of the log groups" from various Lambda functions.
2. **Powerful Query Language:** It provides a robust query language with commands like filter, parse, stats, and sort. The stats command is perfect for aggregating data, and the count function within stats allows for counting events.
3. **Grouping by Type:** To group errors by type, an engineer would typically use the parse command to extract the error type from the log messages (e.g., using regex or common delimiters). Then, the stats count() by *errorType* command can be used to achieve the desired aggregation. For instance, a query might look like *filter @message like /ERROR/ | parse @message "Error Type:" as errorType | stats count() by errorType | sort by count() desc*.
4. **Efficiency and Cost-Effectiveness:** It eliminates the need for complex data pipelines or external services, providing a direct and efficient way to query logs where they reside. Queries are performed on demand, and you only pay for the log data scanned.

Let's briefly examine why the other options are less suitable:

B. Perform a CloudWatch Logs search: The standard CloudWatch Logs search interface offers basic filtering but lacks the advanced aggregation, stats command, and group by capabilities of Logs Insights across multiple log groups that are necessary to categorize and count errors by type.

C. Perform an Amazon Athena query: While Amazon Athena is excellent for querying data in S3 using SQL, it would require first exporting the CloudWatch Logs data to an S3 bucket (e.g., via Kinesis Firehose). This adds additional architectural complexity, latency, and cost for data transport and storage, when a native solution already exists for direct querying.

D. Perform an Amazon RDS query: Amazon RDS is a relational database service unsuitable for direct ingestion and analysis of high-volume, typically semi-structured log data. It would be highly inefficient, costly, and require significant custom development to parse and store logs, making it an entirely inappropriate choice for this requirement.

In conclusion, CloudWatch Logs Insights is the most direct, efficient, and native AWS service to perform complex log analysis, including counting and grouping errors by type across multiple log groups, making it the ideal solution for the CloudOps engineer.

Authoritative Links for further research:

Amazon CloudWatch Logs Insights:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AnalyzingLogData.html>

CloudWatch Logs Insights Query Syntax:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/CWL_QuerySyntax.html

Question: 51

A developer uses an Amazon Linux Amazon Machine Image (AMI) to launch an EC2 instance that hosts a third-party application. The application occasionally becomes unstable. The CloudOps engineer needs a solution to automatically reboot the EC2 instance whenever utilization is above 90% for 15 minutes and to notify the developer about the reboot.

Which solution will meet these requirements with the LEAST administrative effort?

- A. Configure an Amazon CloudWatch alarm that evaluates the CPU utilization of the instance. Configure the alarm to invoke an AWS Lambda function to publish a message to an Amazon Simple Notification Service (Amazon SNS) topic when the CloudWatch alarm activates. Configure the Lambda function to reboot the EC2 instance. Subscribe the developer to the SNS topic.
- B. Create an Amazon CloudWatch alarm that evaluates the CPU utilization of the instance. Configure the alarm to publish a notification to an Amazon Simple Notification Service (Amazon SNS) topic and to perform an EC2 action to reboot the instance. Subscribe the developer to the SNS topic.
- C. Create an Amazon CloudWatch alarm that evaluates the CPU utilization of the instance. Configure the alarm to invoke an AWS Systems Manager action to create an incident to notify the developer and request the reboot.
- D. Create an AWS Systems Manager runbook script to publish a message to an Amazon Simple Notification Service (Amazon SNS) topic and to reboot the EC2 instance. Subscribe the developer to the SNS topic. Configure an Amazon CloudWatch alarm to run the Systems Manager runbook when CPU utilization for the instance remains above 90% for more than 15 minutes.

Answer: B

Explanation:

The correct answer is **B** because it leverages the native capabilities of Amazon CloudWatch alarms to directly perform both the required actions — rebooting the EC2 instance and sending a notification — with the absolute least administrative effort.

Here's a detailed justification:

1. **CloudWatch Alarm for Monitoring:** Amazon CloudWatch is AWS's monitoring service that collects monitoring and operational data in the form of logs, metrics, and events. It's the ideal service for evaluating the CPU utilization metric of an EC2 instance. A CloudWatch alarm can be configured to trigger when the CPU utilization metric is consistently above 90% for 15 minutes, directly meeting the triggering condition.
2. **Native EC2 Action for Reboot:** CloudWatch alarms have built-in actions that can be directly configured to manage EC2 instances. One of these native actions is "Reboot this instance." This eliminates the need for any custom scripting or Lambda functions to perform the reboot. When the alarm state changes from OK to ALARM, CloudWatch can automatically initiate the EC2 reboot, fulfilling the first critical requirement efficiently.
3. **Native SNS Notification:** Another direct action available for CloudWatch alarms is publishing a message to an Amazon Simple Notification Service (Amazon SNS) topic. By configuring the alarm to publish to an SNS topic upon activation, the developer, who is subscribed to that topic, will receive an immediate notification about the instance reboot. This satisfies the second requirement without needing intermediary services like AWS Lambda to send the notification.
4. **Least Administrative Effort:** The core advantage of Option B lies in its reliance on these direct, pre-built integrations within CloudWatch. There is no custom code to write, debug, deploy, or maintain (unlike options involving AWS Lambda functions or AWS Systems Manager runbook scripts). All configurations are performed directly within the CloudWatch alarm setup, significantly reducing development time, operational overhead, and the potential for errors.

5. Comparison with Other Options:

Option A requires writing and maintaining an AWS Lambda function, which introduces administrative overhead for development, security (IAM roles), and ongoing maintenance.

Option C suggests using AWS Systems Manager to create an incident and "request" a reboot, which implies it might not be fully automatic or could be overly complex for a simple automatic reboot.

Option D involves creating an AWS Systems Manager runbook script, similar to Lambda, which adds custom script development and maintenance effort.

In conclusion, Option B provides the most streamlined and efficient solution by leveraging the powerful and integrated capabilities of CloudWatch alarms to directly perform both the EC2 reboot and SNS notification, making it the clear choice for meeting the requirements with the least administrative effort.

Authoritative Links for Further Research:

Using Amazon CloudWatch Alarms:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html>

Stop, Terminate, Reboot, or Recover Your EC2 Instance:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/ec2-actions-alarms.html>

Amazon SNS Integration with CloudWatch: <https://docs.aws.amazon.com/sns/latest/dg/sns-cw-as-integration.html>

Question: 52

CertyIQ

A company has a microservice that runs on a set of Amazon EC2 instances. The EC2 instances run behind an Application Load Balancer (ALB). A CloudOps engineer must use Amazon Route 53 to create a record that maps the ALB URL to example.com. Which type of record will meet this requirement?

- A. An A record
- B. An AAAA record
- C. An alias record
- D. A CNAME record

Answer: C

Explanation:

To map a custom domain like example.com to an AWS Application Load Balancer (ALB) URL using Amazon Route 53, an **alias record** is the most appropriate and recommended solution. Alias records are a Route 53-specific extension to standard DNS functionality, designed to simplify linking domain names to various AWS resources efficiently and reliably.

Unlike traditional DNS records that resolve to static IP addresses, an alias record points directly to the DNS name of another AWS resource, such as an ALB, CloudFront distribution, or S3 bucket. A primary advantage is its ability to automatically track and resolve to the underlying IP addresses of the target AWS resource, even if those IPs change dynamically, which is a common characteristic for ALBs. This automation eliminates the operational burden of manually updating DNS records when an ALB's IP addresses fluctuate, ensuring continuous service availability.

Furthermore, alias records can be created for the zone apex (the root domain, e.g., example.com), which is a critical capability for main website addresses. Route 53 also provides built-in health checks for alias targets, enhancing reliability by automatically routing traffic away from unhealthy endpoints. Additionally, DNS

queries to alias records pointing to AWS resources are free of charge, offering a cost benefit compared to standard DNS queries.

Option A, an A record, maps a hostname to an IPv4 address. This is unsuitable because ALBs have dynamic IP addresses that can change at any time, making manual maintenance of A records impractical and highly prone to service disruption. Similarly, Option B, an AAAA record, maps a hostname to an IPv6 address and suffers from the same dynamic IP challenge.

Option D, a CNAME record (Canonical Name), maps one domain name to another. While a CNAME record could theoretically point to an ALB's DNS name for a subdomain (e.g., `www.example.com`), it cannot be used for the zone apex (`example.com`). Standard DNS specifications prevent CNAMEs at the zone apex because a CNAME declares that the domain is an alias for another, precluding other necessary record types like MX (mail exchanger) or NS (name server) records that must exist at the root. Therefore, using a CNAME for `example.com` would violate DNS standards and prevent other essential services from functioning correctly.

Given the requirement to map `example.com` to an ALB and leverage AWS's managed infrastructure benefits, the alias record is the definitive choice for its flexibility, automation, compatibility with the zone apex, and built-in resilience.

For further research:

AWS Route 53 Developer Guide:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/Welcome.html>

About Alias Records in Route 53:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/CreatingAliasR53Records.html>

Question: 53

CertyIQ

An application runs on Amazon EC2 instances behind an Application Load Balancer (ALB). The application takes up to 2 minutes to populate a local cache after the application is started. The application reports as healthy in the target group health check a few seconds after starting.

A CloudOps engineer observes that after some of the instances are rebooted, the instances receive an equal share of the traffic immediately after each instance reports as healthy. The application needs to receive a gradually increasing share of the traffic while the application cache is populated.

Which solution will meet this requirement?

- A. Change the `slow_start.duration_seconds` target group attribute to 120 seconds. Before rebooting the instances, deregister the instances from the target group. After rebooting the instances, register the instances with the target group.
- B. Change the `HealthCheckTimeoutSeconds` paramotor in the target group to 120 seconds. Before rebooting the instances, deregister the instances from the target group. After rebooting the instances, register the instances with the target group.
- C. Configure an Amazon CloudWatch alarm to monitor the health check status. Configure the action of the alarm to restart an EC2 instance if a health check fails. Change the `loadbalancing.algorithm.type` target group attribute to be `weighted_random`.
- D. Create an Amazon EC2 Auto Scaling group. Attach the existing EC2 instances to the Auto Scaling group. Configure an EC2 Auto Scaling lifecycle hook to move starting instances to the Pending Wait state. Update the application to complete the lifecycle hook when the local cache has been populated.

Answer: A

Explanation:

The correct solution is **A** because it directly addresses the requirement for gradually increasing traffic to

instances that are healthy but still initializing.

Here's a detailed justification:

1. **Problem Identification:** The core issue is that EC2 instances become healthy and start receiving an equal share of traffic before their local cache is fully populated, which takes 2 minutes. This leads to performance issues as the application is not ready for full load immediately after reporting healthy. The requirement is a "gradually increasing share of the traffic" during this 2-minute warm-up period.
2. **Analysis of Option A:**

Change the `slow_start.duration_seconds` target group attribute to 120 seconds. This is the key component of the solution. The Application Load Balancer (ALB) `slow_start.duration_seconds` attribute is specifically designed to handle this exact scenario. When an instance is registered with a target group or becomes healthy after being unhealthy, it receives a gradually increasing share of traffic over the specified duration. Setting this to 120 seconds (2 minutes) perfectly matches the application's cache population time, allowing it to warm up gracefully.

Before rebooting the instances, deregister the instances from the target group. Deregistering instances ensures they are removed from active traffic distribution before they are rebooted. This prevents the ALB from sending requests to an instance that is shutting down or restarting.

After rebooting the instances, register the instances with the target group. Re-registering the instances with the target group marks them as new or re-entering service. Upon becoming healthy, the ALB will then apply the `slow_start.duration_seconds` attribute, gradually increasing the traffic to these instances over the next 120 seconds. This ensures that by the time the instance receives its full share of traffic, its cache is fully populated.

1. Why Other Options Are Incorrect:

B. Change the `HealthCheckTimeoutSeconds` parameter in the target group to 120 seconds.

`HealthCheckTimeoutSeconds` defines how long the ALB waits for a response from a target before declaring the health check a failure. Increasing this value would only delay marking an unresponsive instance as unhealthy; it would not introduce gradual traffic distribution for a healthy instance that is still warming up. The application reports healthy in a few seconds, meaning it responds within the default timeout, but isn't ready for full load. This option does not solve the gradual traffic requirement.

C. Configure an Amazon CloudWatch alarm to monitor the health check status. Configure the action of the alarm to restart an EC2 instance if a health check fails. Change the `loadbalancing.algorithm.type` target group attribute to be `weighted_random`. This option proposes reactive measures for failed health checks (restarting instances) which is not related to gradual warm-up. Changing the load balancing algorithm to `weighted_random` would allow static weighting of targets, but it doesn't dynamically adjust weights over time for a warming-up instance. There's no inherent mechanism in `weighted_random` to automatically ramp up traffic for new or rejoining instances.

D. Create an Amazon EC2 Auto Scaling group. Attach the existing EC2 instances to the Auto Scaling group. Configure an EC2 Auto Scaling lifecycle hook to move starting instances to the Pending Wait state. Update the application to complete the lifecycle hook when the local cache has been populated. While lifecycle hooks are powerful for performing actions during instance launch or termination, they only delay an instance from becoming `InService` and joining the target group until the hook completes. Once the lifecycle hook is completed and the instance is marked `InService`, it would receive full traffic immediately. It ensures readiness before receiving any traffic, but it does not provide a "gradually increasing share of the traffic" over the 2-minute period, which is a specific requirement of the problem.

Conclusion: Option A is the most appropriate and direct solution because the `slow_start.duration_seconds` attribute of an ALB target group is purpose-built for scenarios where instances need a warm-up period after becoming healthy before they can handle full production traffic. The process of deregistering and re-registering ensures that this slow start mechanism is correctly applied each time an instance is brought back online.

Authoritative Links for Further Research:

Target Group Attributes - Slow Start:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html#target-group-attributes> (Look for `slow_start.duration_seconds`)

Register and Deregister Targets:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-register-targets.html>

Question: 54

CertyIQ

A company runs an application on Amazon EC2 instances. The application stores and retrieves data from an Amazon Aurora PostgreSQL database. A developer accidentally drops a table from the database, which causes application errors. Two hours later, a CloudOps engineer needs to recover the data and make the application function again.

Which solution will meet this requirement?

- A. Use the Aurora Backtrack feature to rewind the database to a specified time, 2 hours in the past.
- B. Perform a point-in-time recovery on the existing database to restore the database to a specified point in time, 2 hours in the past.
- C. Perform a point-in-time recovery and create a new database to restore the database to a specified point in time, 2 hours in the past. Reconfigure the application to use a new database endpoint.
- D. Create a new Aurora cluster. Choose the Restore data from S3 bucket option. Choose log files up to the failure time 2 hours in the past.

Answer: C

Explanation:

The correct solution is C because it leverages Aurora's native, robust Point-in-Time Recovery (PITR) capability, which is the standard and most efficient method for recovering from logical data corruption or accidental data loss events like a dropped table.

Here's a detailed justification:

1. **Understanding the Problem:** An accidental table drop constitutes logical data corruption. The goal is to restore the database to a state before this event occurred, within a 2-hour window. The database is Aurora PostgreSQL.
2. **Analysis of Option C (Correct Solution):**

Perform a point-in-time recovery: Amazon Aurora automatically and continuously backs up your cluster volume, storing transaction logs. This enables recovery to any specified second within the backup retention period (typically 1-35 days, up to 5 minutes prior to the current time). This is precisely what's needed for a 2-hour old incident.

Create a new database: A fundamental aspect of Aurora's PITR is that it always creates a new Aurora DB cluster. It achieves this by taking the nearest automatic snapshot and then applying transaction logs to roll forward the database state to the exact specified point in time. It does not modify the existing database cluster.

Restore the database to a specified point in time, 2 hours in the past: This directly addresses the requirement to revert to the state before the table was dropped.

Reconfigure the application to use a new database endpoint: Since PITR creates a new database cluster, this new cluster will have its own unique cluster endpoint. The application, which was previously connected to the old (corrupted) database, must be updated to point to this new endpoint.

to access the recovered, functional data. This is a critical final step to restore application functionality.

3. Why other options are incorrect:

A. Use the Aurora Backtrack feature to rewind the database to a specified time, 2 hours in the past.

Incorrect: Aurora Backtrack is a feature only available for Amazon Aurora MySQL-compatible edition. The question explicitly states "Amazon Aurora PostgreSQL database." Therefore, Backtrack cannot be used.

B. Perform a point-in-time recovery on the existing database to restore the database to a specified point in time, 2 hours in the past.

Incorrect: As explained, Aurora's PITR always creates a new database cluster. It does not perform an in-place recovery on the existing database. Attempting to restore "on the existing database" is a misunderstanding of how Aurora PITR works. The original database cluster remains untouched, allowing for forensic analysis if needed.

D. Create a new Aurora cluster. Choose the Restore data from S3 bucket option. Choose log files up to the failure time 2 hours in the past.

Incorrect: While you can restore data into an Aurora cluster from an S3 bucket (e.g., from a manual backup or data migration), this is not the standard or most efficient method for performing a point-in-time recovery of an existing Aurora database using its native continuous backup mechanism. Aurora's PITR feature (described in option C) automatically handles the process of applying transaction logs from its continuous backup stream, without the user needing to manually specify "log files from S3." This option misrepresents the primary PITR process and describes a less direct, more complex, and potentially less accurate recovery method for this scenario.

Conclusion: Option C accurately describes the correct procedure for performing a point-in-time recovery in Amazon Aurora PostgreSQL, ensuring data integrity and application functionality are restored efficiently.

Authoritative Links for Further Research:

Restoring an Amazon Aurora DB cluster to a specified

time:https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/USER_RestoreFromPointInTime.html

Backtracking an Aurora DB cluster (note the engine

compatibility):<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQL.Managing.Backtrack.html>

Migrating data from Amazon S3 to an Amazon Aurora PostgreSQL DB

cluster:<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/PostgreSQL.S3Import.html>

Question: 55

CertyIQ

A company runs several workloads on AWS. The company identifies five AWS Trusted Advisor service quota metrics to monitor in a specific AWS Region. The company wants to receive email notification each time resource usage exceeds 60% of one of the service quotas.

Which solution will meet these requirements?

A. Create five Amazon CloudWatch alarms, one for each Trusted Advisor service quota metric. Configure an Amazon Simple Notification Service (Amazon SNS) topic for email notification each time that usage exceeds 60% of one of the service quotas.

B. Create five Amazon CloudWatch alarms, one for each Trusted Advisor service quota metric. Configure an Amazon Simple Queue Service (Amazon SQS) queue for email notification each time that usage exceeds 60% of one of the service quotas.

C. Use the AWS Health Dashboard to monitor each Trusted Advisor service quota metric. Configure an Amazon Simple Queue Service (Amazon SQS) queue for email notification each time that usage exceeds 60% of one of the service quotas.

D. Use the AWS Health Dashboard to monitor each Trusted Advisor service quota metric. Configure an Amazon Simple Notification Service (Amazon SNS) topic for email notification each time that usage exceeds 60% of one of the service quotas.

Answer: A

Explanation:

This solution leverages Amazon CloudWatch alarms for precise monitoring and Amazon Simple Notification Service (SNS) for direct email alerts, perfectly aligning with the requirements.

1. **Monitoring Service Quota Metrics with CloudWatch Alarms:** AWS provides metrics for service quotas through Amazon CloudWatch, specifically under the AWS/Usage namespace. These metrics allow users to track current resource utilization against their service quotas. Trusted Advisor also surfaces insights based on these quotas. By creating five separate CloudWatch alarms, one for each identified service quota metric, the company can granularly monitor the usage of each specific resource.
2. **Setting Custom Thresholds (60%):** CloudWatch alarms are designed to watch a single metric over a specified time period and trigger actions when the metric breaches a defined threshold. For this scenario, each alarm can be configured to transition into an ALARM state when the resource usage metric exceeds 60% of its corresponding service quota. This directly addresses the requirement for a 60% usage threshold.
3. **Email Notifications via Amazon SNS:** Amazon Simple Notification Service (SNS) is a fully managed pub/sub messaging service that supports various notification protocols, including email. When a CloudWatch alarm changes its state to ALARM (e.g., usage surpasses 60%), it can publish a message to a designated SNS topic. Any email addresses subscribed to that SNS topic will then receive an immediate email notification, fulfilling the requirement for email alerts.
4. **Scalability and Efficiency:** Creating five distinct alarms ensures independent monitoring for each critical service quota. This setup is highly efficient and scalable within a specific AWS Region, as CloudWatch and SNS are native AWS services designed for robust monitoring and notification workflows.

Why other options are incorrect:

Options B and C (Amazon SQS): Amazon Simple Queue Service (SQS) is a message queuing service primarily used for decoupling application components and asynchronous processing. While SQS can receive messages from CloudWatch alarms or AWS Health events, it does not inherently send email notifications directly. An additional processing layer, such as an AWS Lambda function, would be required to consume messages from SQS and then send emails via SNS or SES, adding unnecessary complexity for a direct email notification requirement.

Options C and D (AWS Health Dashboard): The AWS Health Dashboard provides a personalized view of the health of AWS services and resources, notifying users about events that can affect their AWS environment, including service limit increases or potential service impacts. However, it is not primarily designed for continuous, proactive monitoring of custom utilization percentages like "60% of service quota." While AWS Health can report on service limit breaches, it doesn't allow for the configuration of arbitrary, user-defined thresholds for resource usage against quotas in the same way CloudWatch metrics and alarms do. CloudWatch is the appropriate service for defining and monitoring such specific usage thresholds.

Therefore, creating CloudWatch alarms for each service quota metric and linking them to an SNS topic for email notifications is the most direct, efficient, and native AWS solution.

Authoritative Links for further research:

Monitoring Service Quotas with CloudWatch: AWS Service Quotas documentation on monitoring:

<https://docs.aws.amazon.com/servicequotas/latest/userguide/monitoring-cloudwatch.html>

Creating CloudWatch Alarms: Amazon CloudWatch user guide on creating alarms:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html>

Amazon SNS for Email Notifications: Amazon SNS developer guide on email subscriptions:

<https://docs.aws.amazon.com/sns/latest/dg/sns-email-notifications.html>

Question: 56

CertyIQ

A CloudOps engineer is troubleshooting an implementation of Amazon CloudWatch Synthetics. The CloudWatch Synthetics results must be sent to an Amazon S3 bucket.

The CloudOps engineer has copied the configuration of an existing canary that runs on a VPC that has an internet gateway attached. However, the CloudOps engineer cannot get the canary to successfully start on a private VPC that has no internet access.

What should the CloudOps engineer do to successfully run the canary on the private VPC?

- A. Ensure that the DNS resolution option and the DNS hostnames option are turned on in the VPC. Add the synthetics:GetCanaryRuns permission to the VPC. On the S3 bucket, add the IgnorePublicAcls permission to the CloudWatch Synthetics role.
- B. Ensure that the DNS resolution option and the DNS hostnames option are turned off in the VPC. Create a gateway VPC endpoint for Amazon S3. Add the permissions to allow CloudWatch Synthetics to use the S3 endpoint.
- C. Ensure that the DNS resolution option and the DNS hostnames option are turned off in the VPC. Add a security group to the canary to allow outbound traffic on the DNS port. Add the permissions to allow CloudWatch Synthetics to write to the S3 bucket.
- D. Ensure that the DNS resolution option and the DNS hostnames option are turned on in the VPC. Create an interface VPC endpoint for CloudWatch. Create a gateway VPC endpoint for Amazon S3. Add the permissions to allow CloudWatch Synthetics to use both endpoints.

Answer: D

Explanation:

To successfully run a CloudWatch Synthetics canary on a private VPC without internet access and send results to an Amazon S3 bucket, several key configurations are necessary:

1. **Enable DNS Resolution in the VPC:** Even when using VPC endpoints, DNS resolution is crucial. When the canary tries to connect to AWS services like CloudWatch or S3, it typically attempts to resolve their public DNS names. With `enableDnsSupport` and `enableDnsHostnames` turned on in the VPC, the canary can resolve these service endpoints, and the presence of VPC endpoints will route this traffic privately. Without DNS resolution, the canary would fail to locate the service endpoints.

Authoritative Link: [DNS in your VPC - Amazon Virtual Private Cloud](#)

1. **Create an Interface VPC Endpoint for CloudWatch:** CloudWatch Synthetics is a feature of Amazon CloudWatch. The canary execution environment needs to communicate with the CloudWatch service to report metrics, logs, and potentially its status. Since the VPC is private and lacks internet access, an interface VPC endpoint (powered by AWS PrivateLink) for CloudWatch is essential. This allows the canary to communicate securely and privately with the CloudWatch API within the AWS network, bypassing the public internet.

Authoritative Link: [Using CloudWatch Synthetics with a VPC - AWS CloudWatch Synthetics](#)

Authoritative Link: [CloudWatch endpoints and quotas - AWS General Reference](#)

1. **Create a Gateway VPC Endpoint for Amazon S3:** The problem explicitly states that "The

CloudWatch Synthetics results must be sent to an Amazon S3 bucket." To allow the canary to store its results in S3 from a private VPC without an internet gateway or NAT gateway, a gateway VPC endpoint for Amazon S3 is required. Gateway endpoints provide reliable and private connectivity from your VPC to S3.

Authoritative Link: [Gateway VPC endpoints - Amazon Virtual Private Cloud](#)

1. **Add Permissions for Endpoint Usage:** While the CloudWatch Synthetics execution role will need permissions to perform actions (e.g., `s3:PutObject` for S3, `cloudwatch:PutMetricData` for CloudWatch), it's also crucial to ensure that the VPC endpoint policies allow the CloudWatch Synthetics role to utilize these endpoints. This is a security best practice to control who can access the services through the private endpoints.

Option D correctly identifies all these critical components, making it the comprehensive solution for running a CloudWatch Synthetics canary in a private VPC. Other options fail to address either the DNS requirements, the need for both CloudWatch and S3 endpoints, or propose incorrect or insufficient configurations.

Question: 57

CertyIQ

A company deploys AWS infrastructure in a VPC that has an internet gateway. The VPC has public subnets and private subnets. An Amazon RDS for MySQL DB instance is deployed in a private subnet. An AWS Lambda function uses the same private subnet and connects to the DB instance to query data.

A developer modifies the Lambda function to require the function to publish messages to an Amazon Simple Queue Service (Amazon SQS) queue. After these changes, the Lambda function times out when it tries to publish messages to the SQS queue.

Which solutions will resolve this issue? (Choose two.)

- A. Reconfigure the Lambda function so that the function is not connected to the VPC.
- B. Deploy an RDS proxy. Configure the Lambda function to connect to the DB instance through the proxy.
- C. Deploy a NAT gateway. Update the private subnet's route table to route all traffic to the NAT gateway.
- D. Create an interface endpoint for Amazon SQS in the VPC.
- E. Create a gateway endpoint for Amazon SQS in the VPC.

Answer: CD

Explanation:

The core problem is that the AWS Lambda function, deployed within a private subnet of a VPC, can no longer reach Amazon SQS after being modified. While it could previously connect to the Amazon RDS instance in the same private subnet (due to internal VPC communication), SQS is a public AWS service endpoint that requires outbound network access for the Lambda function to communicate with it. Resources in a private subnet do not have direct internet access by default.

Here's why options C and D are correct:

C. Deploy a NAT gateway. Update the private subnet's route table to route all traffic to the NAT gateway. A NAT Gateway enables instances in a private subnet to connect to the internet or other AWS services outside the VPC, while preventing those services from initiating connections back to the instances. To implement this, a NAT Gateway is deployed in a public subnet, which has a route to the Internet Gateway. The private subnet's route table is then updated to direct all outbound traffic (typically `0.0.0.0/0`) to the NAT Gateway. This provides the necessary path for the Lambda function to reach the public SQS endpoint.

Reference: [AWS NAT Gateways](#)

D. Create an interface endpoint for Amazon SQS in the VPC. An interface endpoint, powered by AWS PrivateLink, allows you to establish a private connection from your VPC to supported AWS services, including Amazon SQS. When an interface endpoint for SQS is created within the VPC, the Lambda function in the private subnet can access SQS using private IP addresses. This keeps traffic entirely within the AWS network, bypassing the need for an Internet Gateway or a NAT Gateway for that specific service, offering a more secure and often more performant solution.

Reference: [VPC Interface Endpoints \(AWS PrivateLink\)](#)

Here's why the other options are incorrect:

A. Reconfigure the Lambda function so that the function is not connected to the VPC. While this would give the Lambda function internet access to reach SQS, it would also disconnect it from the VPC. Consequently, the Lambda function would lose its ability to connect to the Amazon RDS instance, which is deployed in a private subnet within that specific VPC, thus breaking existing functionality required by the problem statement.

B. Deploy an RDS proxy. Configure the Lambda function to connect to the DB instance through the proxy.

An RDS proxy helps manage database connections and improve scalability, but it operates within the VPC and is specifically for database connectivity. It does not provide any mechanism for the Lambda function to access external AWS services like SQS. The problem is with SQS connectivity, not RDS.

E. Create a gateway endpoint for Amazon SQS in the VPC. Gateway endpoints are a type of VPC endpoint that are currently only supported for Amazon S3 and Amazon DynamoDB. They are not available for Amazon SQS. For SQS, an interface endpoint (PrivateLink) is the correct type of VPC endpoint to use.

Reference: [VPC Gateway Endpoints](#)

Question: 58

CertyIQ

A company's security policy requires incoming SSH traffic to be restricted to a defined set of addresses. The company is using an AWS Config rule to check whether security groups allow unrestricted incoming SSH traffic. A CloudOps engineer discovers a noncompliant resource and fixes the security group manually. The CloudOps engineer wants to automate the remediation of other noncompliant resources. What is the MOST operationally efficient solution that meets these requirements?

- A. Create an Amazon CloudWatch alarm for the AWS Config rule's status metric. Create an AWS Lambda function that can remove the noncompliant rule from the security group. Configure the alarm action to invoke the Lambda function.
- B. Configure an automatic remediation action on the AWS Config rule. Specify the AWS-DisablnIncomingSSHOnPort22 remediation action.
- C. Configure an Amazon EventBridge rule for AWS Config configuration item change events. Create an AWS Lambda function that can remove the noncompliant rule from the security group. Configure the rule to invoke the Lambda function.
- D. Create an AWS Lambda function that can analyze a security group's inbound rules to check for unrestricted SSH access. Configure the Lambda function to remove the noncompliant rule from the security group. Configure an Amazon EventBridge rule to invoke the Lambda function every hour.

Answer: B

Explanation:

Option B presents the MOST operationally efficient solution because it leverages AWS Config's built-in capabilities for continuous compliance monitoring and automated remediation, requiring minimal custom development and maintenance.

Here's a detailed justification:

1. **Native AWS Config Remediation:** AWS Config allows you to define "remediation actions" directly on a rule. When a resource is flagged as noncompliant, Config can automatically trigger a specified action. This is the intended and most efficient workflow for automating fixes based on Config findings.
2. **Managed Automation Document:** The AWS-DisableIncomingSSHPort22 action is a pre-defined AWS Systems Manager Automation document. This means AWS has already written, tested, and maintains the underlying code to specifically identify and remove inbound SSH rules (port 22) that allow unrestricted access (0.0.0.0/0) from a security group.
3. **Reduced Operational Overhead:** By using a managed automation document, the CloudOps engineer avoids writing, deploying, and maintaining custom Lambda code for the remediation logic. AWS handles updates to the automation document, ensuring it remains compatible with AWS APIs and best practices. This significantly reduces the operational burden compared to custom solutions.
4. **Seamless Integration:** The remediation action is directly linked to the AWS Config rule. When Config detects non-compliance, it can automatically initiate the fix without requiring separate monitoring (like CloudWatch alarms on metrics) or complex event-driven architectures to bridge detection and remediation.
5. **Event-Driven and Precise:** AWS Config continuously evaluates resources. When a security group becomes noncompliant, the remediation action is triggered almost immediately (or within the configured frequency for Config rules), making it event-driven and precise.

Why other options are less efficient:

A. CloudWatch Alarm for Status Metric: An AWS Config rule's "status metric" primarily indicates the rule's operational state (e.g., successful evaluations, errors), not the compliance status of individual resources. You cannot reliably identify which specific resources are noncompliant and trigger remediation directly from a status metric. This approach would be complex to implement and inefficient for specific resource remediation.

C. EventBridge Rule with Custom Lambda: While using EventBridge to trigger a Lambda function on Config compliance changes (specifically AWS Config Rule Compliance Change events) is a valid pattern, it still requires the CloudOps engineer to write and maintain a custom Lambda function to perform the remediation logic. This introduces more development, testing, and maintenance overhead compared to using a pre-built, managed automation document as in Option B.

D. Hourly Lambda Polling: This option suggests bypassing the existing AWS Config rule for detection. It duplicates the detection logic within a custom Lambda function and relies on hourly polling. This is inefficient because the company already has a Config rule for continuous monitoring, and polling is less reactive and potentially more resource-intensive than Config's event-driven evaluations.

Therefore, configuring an automatic remediation action on the AWS Config rule with the AWS-DisableIncomingSSHPort22 managed document is the most operationally efficient solution, leveraging native AWS services for a robust and low-maintenance automation.

Authoritative Links for further research:

AWS Config Remediation Actions: <https://docs.aws.amazon.com/config/latest/developerguide/remediation-actions.html>

AWS Systems Manager Automation documents: <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-automation-documents.html>

AWS-DisableIncomingSSHPort22 documentation (example within Systems Manager): You can find details for this specific document by searching in the AWS Systems Manager console under "Automation" -> "Documents". The specific name is AWS-DisableIncomingSSHPort22.

Question: 59

A CloudOps engineer is maintaining a web application using an Amazon CloudFront web distribution, an

Application Load Balancer (ALB), Amazon RDS, and Amazon EC2 in a VPC. All services have logging enabled. The CloudOps engineer needs to investigate HTTP Layer 7 status codes from the web application. Which log sources contain the status codes? (Choose two.)

- A. VPC Flow Logs
- B. AWS CloudTrail logs
- C. ALB access logs
- D. CloudFront access logs
- E. RDS logs

Answer: CD

Explanation:

The correct log sources that contain HTTP Layer 7 status codes from a web application are ALB access logs and CloudFront access logs.

1. **C. ALB access logs:** Application Load Balancers (ALB) operate at Layer 7 of the OSI model, which is the application layer. ALB access logs capture detailed information about every request sent to the load balancer. Crucially, these logs include the HTTP status code returned by the ALB to the client (elb_status_code) and the HTTP status code returned by the registered targets (e.g., EC2 instances) to the ALB (target_status_code). This makes them a primary source for investigating application-level response statuses.
2. **D. CloudFront access logs:** Amazon CloudFront is a content delivery network (CDN) that also operates at Layer 7. CloudFront access logs provide comprehensive records of all user requests that the CloudFront distribution receives. These logs explicitly contain the HTTP status code (sc-status) returned to the viewer, indicating whether the request was served successfully, resulted in an error, or was redirected. This data is vital for understanding client-side experience and caching behavior.

Why the other options are incorrect:

A. VPC Flow Logs: VPC Flow Logs record metadata about IP traffic (Layer 3 and Layer 4) flowing through network interfaces in a VPC. They capture information like source/destination IP addresses, ports, protocols (e.g., TCP, UDP), and ACCEPT/REJECT status. They do not inspect the application-layer payload and therefore do not contain HTTP status codes.

B. AWS CloudTrail logs: AWS CloudTrail records API calls made to AWS services within your account. It tracks management events, indicating who performed what action, when, and from where. While it shows the success or failure of an AWS API call, it does not log the HTTP status codes generated by your web application when serving content to end-users.

E. RDS logs: Amazon RDS provides various database logs (e.g., error logs, slow query logs, general query logs). These logs contain information related to database operations, queries, and errors. They do not deal with HTTP requests or Layer 7 status codes from a web application.

Therefore, to investigate HTTP Layer 7 status codes, the CloudOps engineer must examine the access logs from both CloudFront (for viewer-facing status) and the ALB (for statuses between the ALB and the backend application).

Authoritative Links for Further Research:

ALB Access Logs: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-access-logs.html>

CloudFront Access Logs:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/AccessLogs.html>

VPC Flow Logs: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

AWS CloudTrail: <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-concepts.html>

Amazon RDS Database Log Files:

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_Logging.html

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

But Wait

I wanted to let you know that there is more content available in the full version. The full paper contains additional sections and information that you may find helpful, and I encourage you to download it to get a more comprehensive and detailed view of all the subject matter.

[Download Full Version Now](#)



Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - certyiqofficial@gmail.com



Free Updates

Lifetime Free Updates!

Total: **195 Questions**

Link: <https://certyiq.com/papers/amazon/aws-certified-cloudops-engineer-associate-soa-c03>